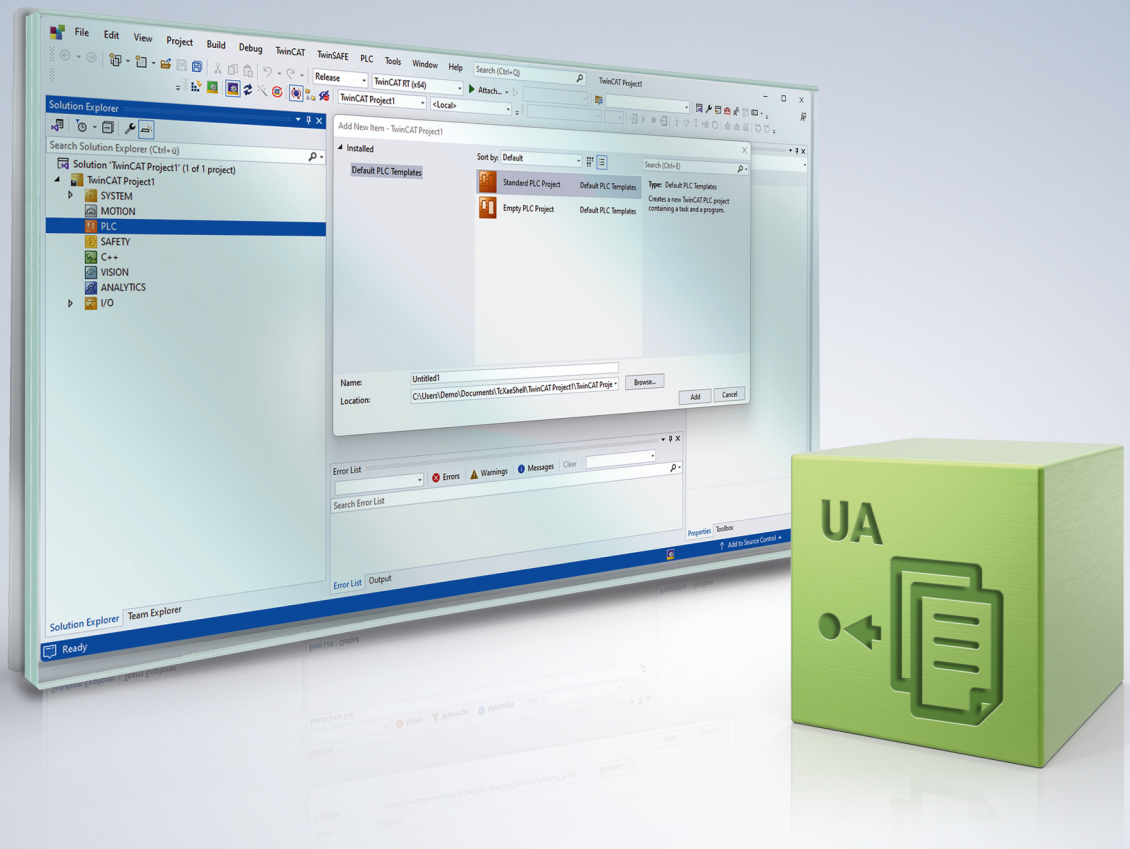
BECKHOFF New Automation Technology

Manual | EN

TE6100

TwinCAT 3 | OPC UA Nodeset Editor



2026-04-28 | Version: 1.3.0

Table of contents

1 Foreword 5
1.1 Notes on the documentation 5
1.2 For your safety 6
1.3 Notes on information security 7
1.4 Documentation output status 7

2 Overview 8

3 Installation 10
3.1 System requirements 10
3.2 Installation 10
3.3 Licensing 11

4 Technical introduction 14
4.1 Quick Start 14
4.1.1 Simple nodeset with base types 14
4.1.2 Simple companion specification 20
4.2 Supported features 25
4.3 License information 26
4.4 User interface 26
4.4.1 Type Explorer 27
4.4.2 Instance Explorer 28
4.4.3 Node Editor 29
4.4.4 Models 29
4.4.5 Menu 30
4.4.6 Message Viewer 31
4.4.7 TwinCAT Target Browser 31
4.4.8 Themes 32
4.5 Modeling 34
4.5.1 Instantiation 34
4.5.2 Arrays 34
4.6 Mapping 35
4.6.1 Data type assignment TwinCAT <=> OPC UA 40
4.7 Application directories 42
4.8 Application and project settings 43
4.9 Exporting a project 46

5 PLC API 49
5.1 Tc3_OpcUa 49
5.1.1 Data types 49
5.1.2 Function blocks 56

6 Samples 58

7 Tutorials 59

8 Appendix 60
8.1 Terminology 60
8.2 Troubleshooting 60

8.3 Usage scenarios and version compatibility 61

9 Support and Service 62

1 Foreword

1.1 Notes on the documentation

This description is intended exclusively for trained specialists in control and automation technology who are familiar with the applicable national standards.
The documentation and the following notes and explanations must be complied with when installing and commissioning the components.
The trained specialists must always use the current valid documentation.

The trained specialists must ensure that the application and use of the products described is in line with all safety requirements, including all relevant laws, regulations, guidelines, and standards.

Disclaimer

The documentation has been compiled with care. The products described are, however, constantly under development.
We reserve the right to revise and change the documentation at any time and without notice.
Claims to modify products that have already been supplied may not be made on the basis of the data, diagrams, and descriptions in this documentation.

Trademarks

Beckhoff®, ATRO®, EtherCAT®, EtherCAT G®, EtherCAT G10®, EtherCAT P®, MX-System®, Safety over EtherCAT®, TC/BSD®, TwinCAT®, TwinCAT/BSD®, TwinSAFE®, XFC®, XPlanar®, and XTS® are registered and licensed trademarks of Beckhoff Automation GmbH.
If third parties make use of the designations or trademarks contained in this publication for their own purposes, this could infringe upon the rights of the owners of the said designations.

EtherCAT® is a registered trademark and patented technology, licensed by Beckhoff Automation GmbH, Germany.

Copyright

© Beckhoff Automation GmbH & Co. KG, Germany.
The distribution and reproduction of this document, as well as the use and communication of its contents without express authorization, are prohibited.
Offenders will be held liable for the payment of damages. All rights reserved in the event that a patent, utility model, or design are registered.

Third-party trademarks

Trademarks of third parties may be used in this documentation. You can find the trademark notices here: https://www.beckhoff.com/trademarks.

1.2 For your safety

Safety regulations

Read the following explanations for your safety.
Always observe and follow product-specific safety instructions, which you may find at the appropriate places in this document.

Exclusion of liability

All the components are supplied in particular hardware and software configurations which are appropriate for the application. Modifications to hardware or software configurations other than those described in the documentation are not permitted, and nullify the liability of Beckhoff Automation GmbH & Co. KG.

Personnel qualification

This description is only intended for trained specialists in control, automation, and drive technology who are familiar with the applicable national standards.

Signal words

The signal words used in the documentation are classified below. In order to prevent injury and damage to persons and property, read and follow the safety and warning notices.

Personal injury warnings

⚠ DANGER
Hazard with high risk of death or serious injury.

⚠ WARNING
Hazard with medium risk of death or serious injury.

⚠ CAUTION
There is a low-risk hazard that could result in medium or minor injury.

Warning of damage to property or environment

NOTICE
The environment, equipment, or data may be damaged.

Information on handling the product



This information includes, for example:
recommendations for action, assistance or further information on the product.

1.3 Notes on information security

The products of Beckhoff Automation GmbH & Co. KG (Beckhoff), insofar as they can be accessed online, are equipped with security functions that support the secure operation of plants, systems, machines and networks. Despite the security functions, the creation, implementation and constant updating of a holistic security concept for the operation are necessary to protect the respective plant, system, machine and networks against cyber threats. The products sold by Beckhoff are only part of the overall security concept. The customer is responsible for preventing unauthorized access by third parties to its equipment, systems, machines and networks. The latter should be connected to the corporate network or the Internet only if appropriate protective measures have been set up.

In addition, the recommendations from Beckhoff regarding appropriate protective measures should be observed. Further information regarding information security and industrial security can be found in our https://www.beckhoff.com/secguide.

Beckhoff products and solutions undergo continuous further development. This also applies to security functions. In light of this continuous further development, Beckhoff expressly recommends that the products are kept up to date at all times and that updates are installed for the products once they have been made available. Using outdated or unsupported product versions can increase the risk of cyber threats.

To stay informed about information security for Beckhoff products, subscribe to the RSS feed at https://www.beckhoff.com/secinfo.

1.4 Documentation output status

Version	Change
1.3.x	**New:** Mapping of an OPC UA structure type as a PLC data type [▶ 53] Instantiation_Boolean [▶ 16] User interface [▶ 26] Modeling [▶ 34] PLC API [▶ 49]
1.2.x	**New:** Data type assignment TwinCAT <=> OPC UA [▶ 40] Usage scenarios and version compatibility [▶ 61]
1.1.x	Overview [▶ 8]: new graphic System requirements [▶ 10]: Number of logical processors supplemented Technical introduction: Graphics replaced Technical introduction/supported features [▶ 25]: Features and descriptions supplemented Technical introduction/user interface/Instance Explorer [▶ 28]: Buttons and actions supplemented Technical introduction/user interface/models [▶ 29]: Graphics replaced, description supplemented Technical introduction/user interface/menu [▶ 30]: Menu entries supplemented Technical introduction/user interface/TwinCAT Target Browser [▶ 31]: Graphic replaced Technical introduction/mapping [▶ 35]: Description and data types, methods supplemented Technical introduction/application and project settings [▶ 42]: Application settings supplemented
1.0.x	First release

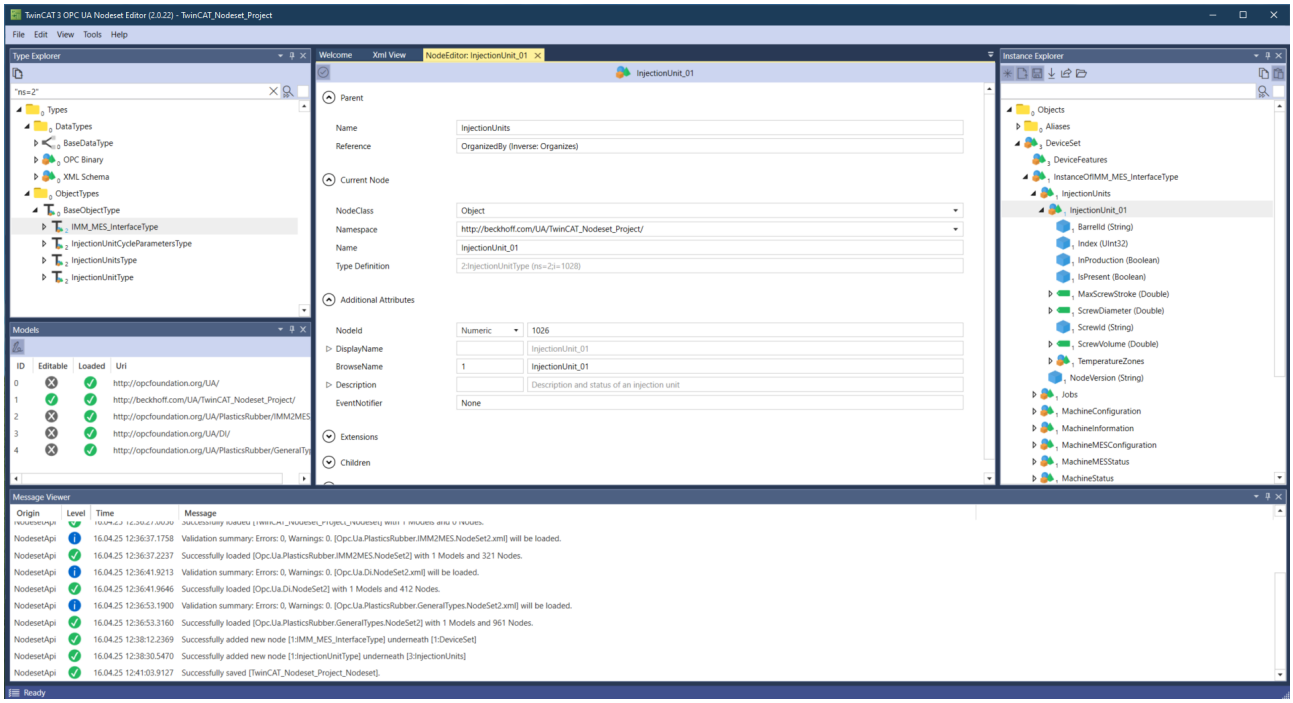
2 Overview

The TwinCAT OPC UA Nodeset Editor is an engineering tool for creating and editing OPC UA nodeset files, which are used for companion specifications and other elements. With the help of the editor, you can map existing companion specifications or your own information models on your Beckhoff controller and link them with data points from the PLC.

When developing a machine application, there are often requirements for the structure of the OPC UA namespace of a machine to be mapped. In this case, it is necessary to establish conformity to a companion specification or a customer-specific information model so that the machine can be integrated as smoothly as possible into an existing and standardized environment.

The TwinCAT OPC UA Nodeset Editor is the graphical solution to meet this challenge. It allows:

- Loading nodeset files from companion specifications or custom information models.
- Creation of simple user-defined nodeset files.
- Convenient linking of objects from the controller with those from a loaded or created nodeset.
- Using the nodeset for new and existing machines using the TwinCAT OPC UA Server (TF6100).



Use cases

The TwinCAT OPC UA Nodeset Editor supports the user in the implementation of requirements regarding companion specifications by providing a graphical user interface that enables the import of nodesets. However, even without requirements for a specific companion specification, the TwinCAT OPC UA Nodeset Editor may be a useful tool and assist in the following use cases.

Creation of custom server address space

Even before companion specifications became widespread, there were initial requests to make information from the PLC available on the TwinCAT OPC UA Server according to a defined structure. By default, the server uses the IEC61131-3 information model to provide variables from the PLC via OPC UA. However, sometimes this data representation on the server is not desired and there were requests for a "simple modeling option for folders and simple data types". This use case can be addressed with the help of this product.

Linking variables and methods from existing nodesets

Nodesets that have been created with a third-party tool, such as the UA Modeler (https://www.unified-automation.com), can be imported into the TwinCAT OPC UA Server. However, since third-party tools do not have any variable information from a TwinCAT control program, the TwinCAT OPC UA Nodeset Editor

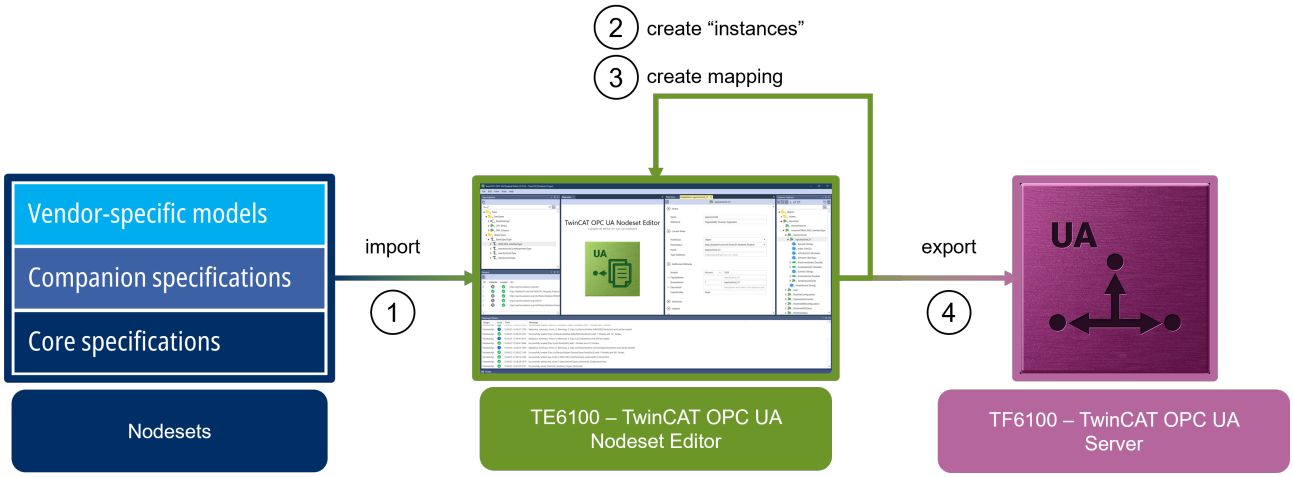
can be used instead. The existing nodesets can be imported into the graphical user interface of the Nodeset Editor and linked there with PLC variables and job methods using the TwinCAT Target Browser. The resulting nodeset file can then be deployed to the TwinCAT OPC UA Server.

Creating instance nodesets from a companion specification

The import of companion specifications and the creation of instances from the data and object types contained therein is required if a machine is to be equipped with a standard interface defined by the companion specifications. The TwinCAT OPC UA Nodeset Editor enables the instantiation of such data and object types and the subsequent linking of the instances with variables and job methods from the TwinCAT PLC.

Workflow

The workflow when using the TwinCAT OPC UA Nodeset Editor to create a nodeset and the subsequent export to the TwinCAT OPC UA Server can be illustrated as follows.



In step (1) the nodeset file (the "companion specification") is imported into the TwinCAT OPC UA Nodeset Editor. This means that the data types defined in the file are available in the editor environment and can be used for project planning.

In step (2) and (3) the project engineering takes place, i.e. objects ("instances") are created from the data types and linked with variables from the PLC.

In step (4) the generated project is exported to the TwinCAT OPC UA Server so that it can provide the generated information model in its address space.

Also see about this

- Supported features [▶ 25]

3 Installation

3.1 System requirements

The following table describes the technical system requirements for the pure operation of the application.

Technical data	Description
Operating system	Windows 10, 11
Target platform	PC architecture (x86, x64)
Number of logical processors	min. 2
Minimum TwinCAT version	TwinCAT 3.1 Build 4026
Required TwinCAT setup level	TwinCAT 3 XAE, XAR, ADS
Required TwinCAT license	TE6100 TC3 OPC UA Nodeset Editor L1 TE6101 TC3 OPC UA Nodeset Editor L2 (optional add-on license)

TF6100 TC3 OPC UA Server

This product works together with the TwinCAT OPC UA Server to run a created Nodeset and display it on the server's address space. Please ensure that the TF6100 TC3 OPC UA Server product is installed locally and/or on your target system.

Even if the Nodeset Editor works together with the TwinCAT OPC UA Server, both products do not have to be installed on the same system. The system requirements for each product apply only to the system on which the product is installed. For more information, see Usage scenarios and version compatibility [▶ 61] .

TE6100 and TF6100 version compatibility

To be able to use the full documented range of functions of the current TwinCAT OPC UA Node Set Editor (TE6100) with the TwinCAT OPC UA Server (TF6100), version 5.2.112 or higher must be installed. The use of an old version is not recommended.

3.2 Installation

If you are using TwinCAT 3.1 Build 4026 (and higher) on the Windows operating system, you can install this function via the TwinCAT Package Manager, see Installation documentation. Normally you install the function via the corresponding workload. However, you can also install the packages contained in the workload individually. This documentation briefly describes the installation process via the workload.

TwinCAT 3 library - Tc3_OpcUa

When installing the editor via TcPkg, the Tc3_OpcUa library is installed. This contains some type definitions that can be used in combination with the editor.

Command line program TcPkg

You can use the TcPkg Command Line Interface (CLI) to display the available workloads on the system:

```
tcpkg list -t workload
```

You can use the following command to install the workload of this function.

```
tcpkg install TE6100.OpcUaNodesetEditor.XAE
```

TwinCAT Package Manager UI

You can use the User Interface (UI) to display all available workloads and install them if required.

To do this, follow the corresponding instructions in the interface.

Installation of the corresponding package or workload can be either done via TcPkg CLI or TcPkg UI.

Unprepared TwinCAT restart can cause data loss

The installation of this function may result in a TwinCAT restart.

Make sure that no critical TwinCAT applications are running on the system or shut them down in an orderly manner first.

3.3 Licensing

The TwinCAT 3 function can be activated as a full version or as a 7-day test version. Both license types can be activated via the TwinCAT 3 development environment (XAE).

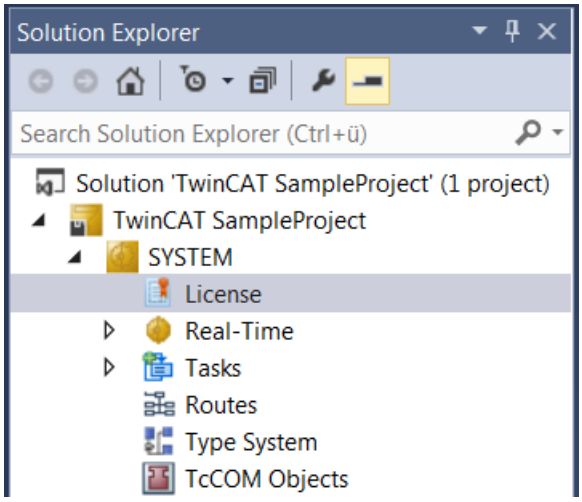
Licensing the full version of a TwinCAT 3 Function

A description of the procedure to license a full version can be found in the Beckhoff Information System in the documentation "TwinCAT 3 Licensing".

Licensing the 7-day test version of a TwinCAT 3 Function

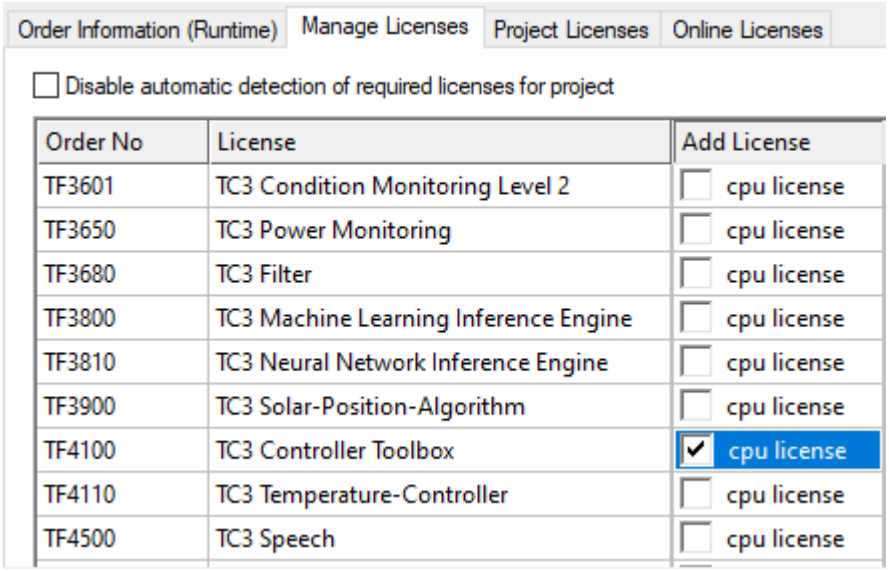
A 7-day test version cannot be enabled for a TwinCAT 3 license dongle.

1. Start the TwinCAT 3 development environment (XAE).
2. Open an existing TwinCAT 3 project or create a new project.
3. If you want to activate the license for a remote device, set the desired target system. To do this, select the target system from the **Choose Target System** drop-down list in the toolbar.
   ⇨ The licensing settings always refer to the selected target system. When the project is activated on the target system, the corresponding TwinCAT 3 licenses are automatically copied to this system.
4. In the **Solution Explorer**, double-click **License** in the **SYSTEM** subtree.

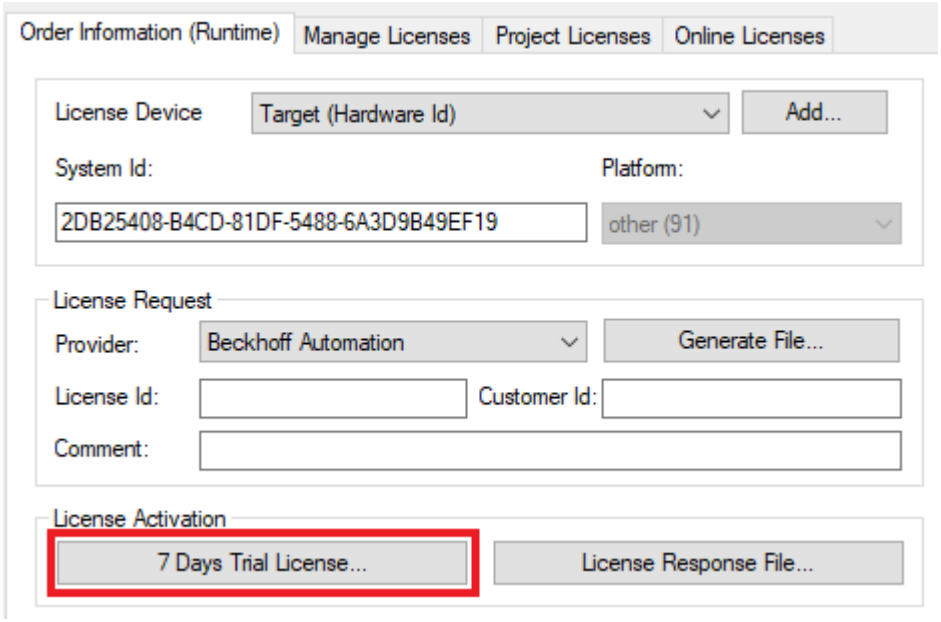


⇨ The TwinCAT 3 license manager opens.

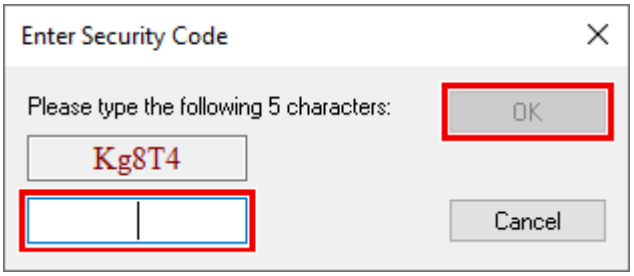
5. Open the **Manage Licenses** tab. In the **Add License** column, check the check box for the license you want to add to your project (e.g. "TF4100 TC3 Controller Toolbox").



6. Open the **Order Information (Runtime)** tab.
   ⇨ In the tabular overview of licenses, the previously selected license is displayed with the status “missing”**.**
7. Click **7-Day Trial License...** to activate the 7-day trial license.



   ⇨ A dialog box opens, prompting you to enter the security code displayed in the dialog.



8. Enter the code exactly as it is displayed and confirm the entry.
9. Confirm the subsequent dialog, which indicates the successful activation.
   ⇨ In the tabular overview of licenses, the license status now indicates the expiry date of the license.

10. Restart the TwinCAT system.
⇨ The 7-day trial version is enabled.

4 Technical introduction

4.1 Quick Start

The following chapter will guide you through some quick start tutorials on how to use the TwinCAT OPC UA Nodeset Editor. Please make sure that you have followed the installation instructions and that your system meets the system requirements. The following quick start tutorials are available:

Tutorial	Description
Simple nodeset with base types [▶ 14]	Demonstrates how to create a nodeset with simple data type instances.
Simple companion specification [▶ 20]	Demonstrates how to create a nodeset with object instances from a companion specification.

4.1.1 Simple nodeset with base types

This documentation article is intended to get you started using this product quickly. After successful installation [▶ 10] and licensing [▶ 11], perform the following steps to create an initial nodeset.

OPC UA Core Nodeset

By default, the TwinCAT OPC UA Nodeset Editor automatically loads the OPC UA Core Nodeset, which defines the namespace http://opcfoundation.org/UA/.

TF6100 TC3 OPC UA Server

This product works together with the TwinCAT OPC UA Server to run a created Nodeset and display it on the server's address space. Please ensure that the TF6100 TC3 OPC UA Server product is installed locally and/or on your target system.

The following Quick Start tutorial will guide you through the following steps:

- Preparation of a PLC project
- Creating a new project
- Creating instances from the OPC UA Core Nodeset
- Linking the created instances with PLC variables
- Deployment of the project on the local TwinCAT OPC UA server

Preparing a PLC project

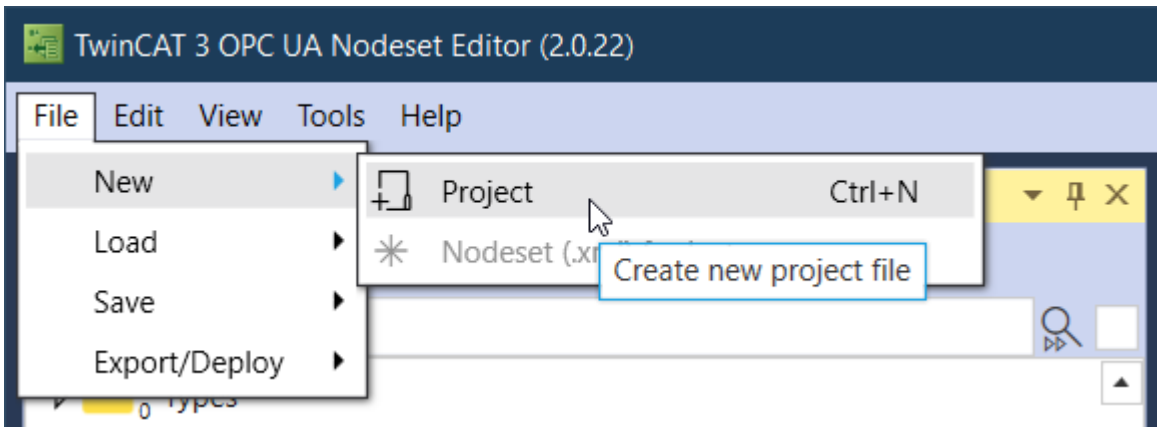
Create a new TwinCAT project and add a PLC project. Create the following data structure in the PLC project and instantiate it three times in the MAIN program.

```
TYPE ST_MyStructure :
STRUCT
  MyBoolean : BOOL;
  MyInt16 : INT;
  MyFloat : REAL;
END_STRUCT
END_TYPE

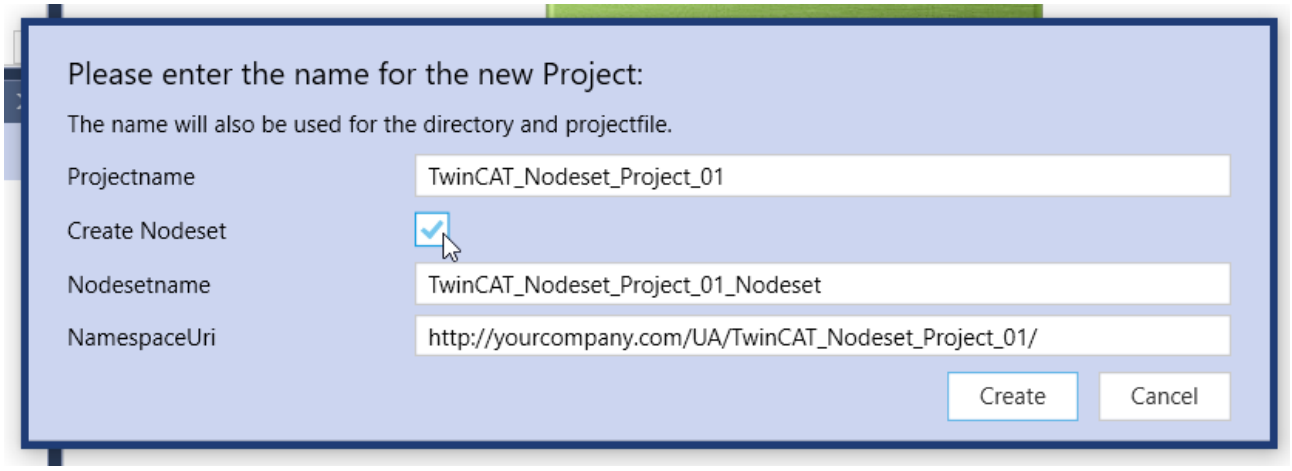
PROGRAM MAIN
VAR
  MyStruct1 : ST_MyStructure;
  MyStruct2 : ST_MyStructure;
  MyStruct3 : ST_MyStructure;
END_VAR
```

Creating a new project

To create a new Nodeset project, please open the menu **File** and click on **New > Project**.



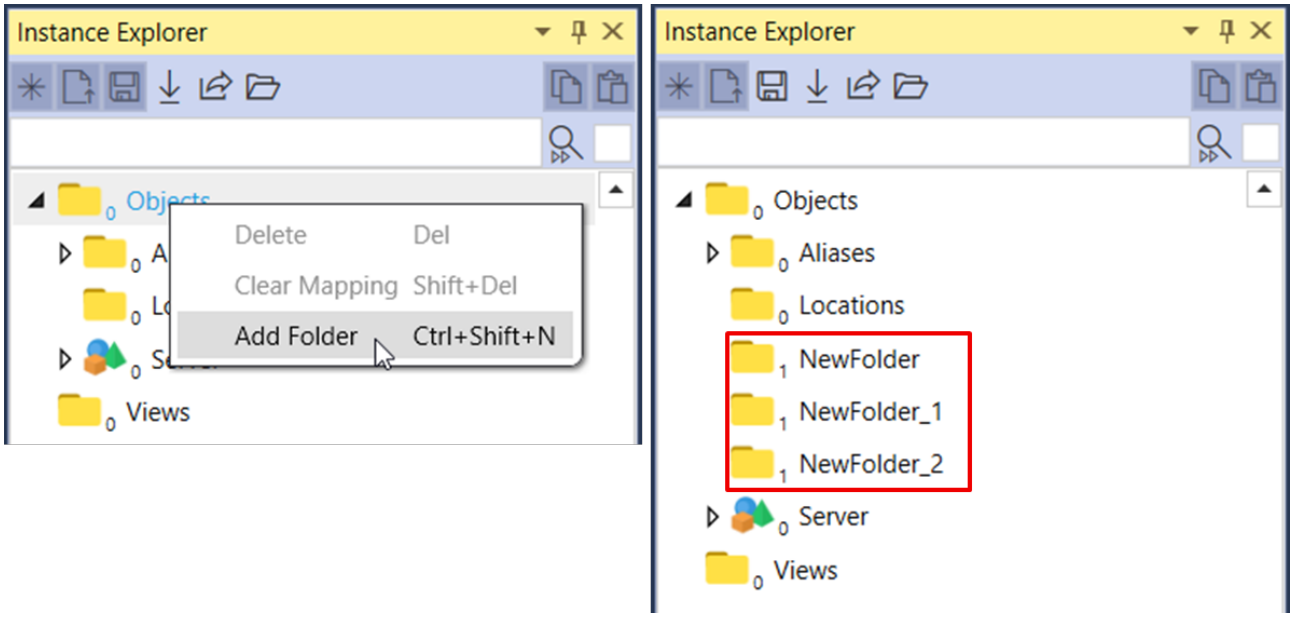
In the following selection dialog, you can configure the name for the project and, if the checkbox is activated (by default), a nodeset to be created. The name and the (namespace) Uri for a new information model can be defined in this nodeset.



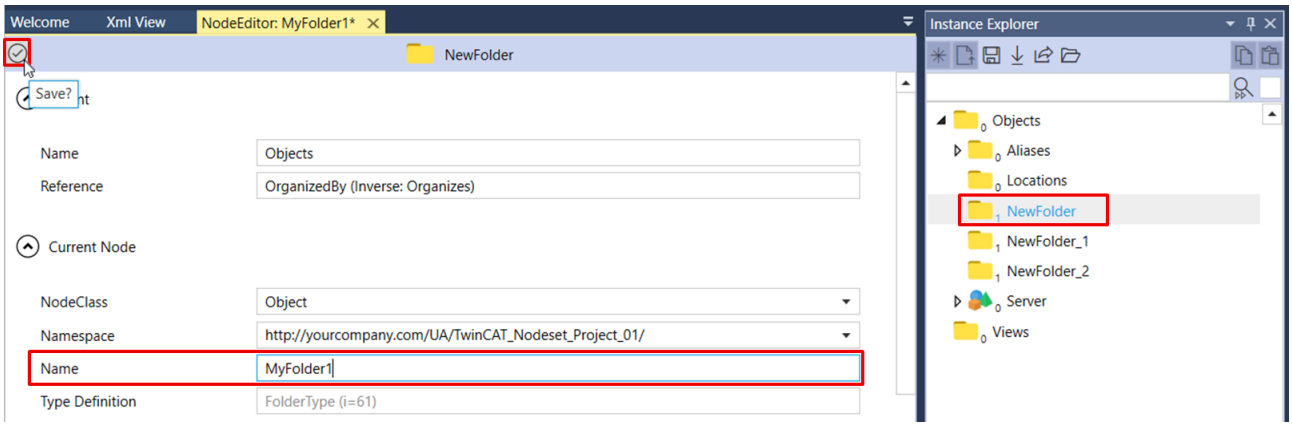
Creating Instances

In this tutorial, we want to create a number of instances with simple data types (Integer, Boolean, Float) and group them in three different folders.

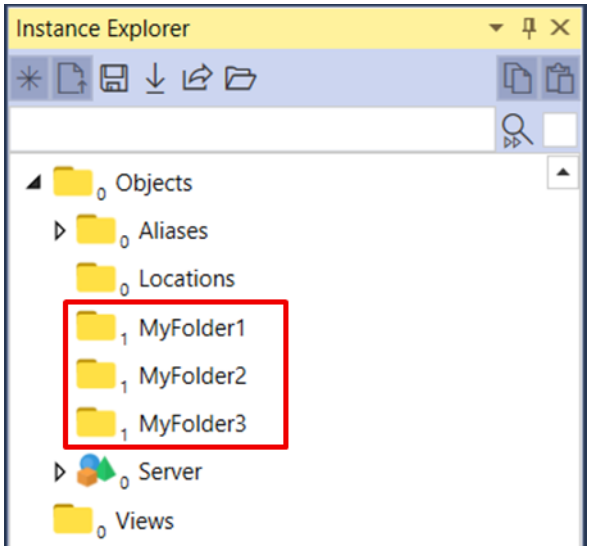
Go to the Instance Explorer and add three folders by selecting the **Objects** node and choosing **Add Folder** from the context menu. The three folders will then be shown as follows:



Rename the folders by double-clicking on each folder and adjusting the **Name** attribute in the Attribute Explorer accordingly. Save this change via the **Save** button on the toolbar.

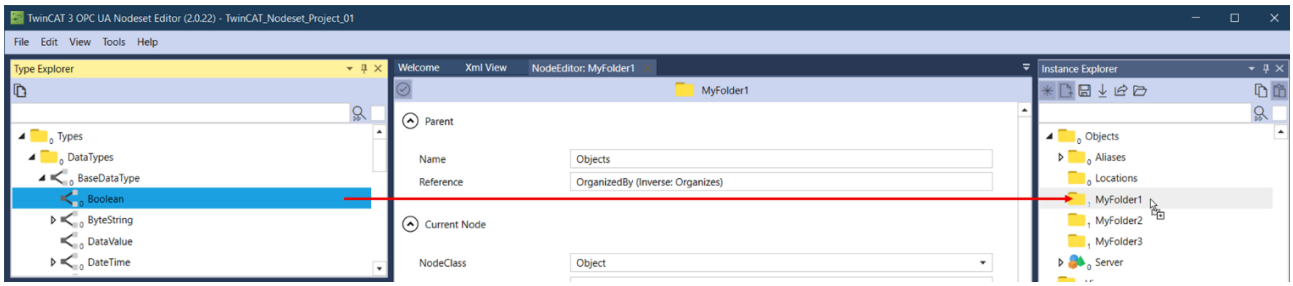


Perform this procedure for all three folders so that the Instance Explorer looks like this:

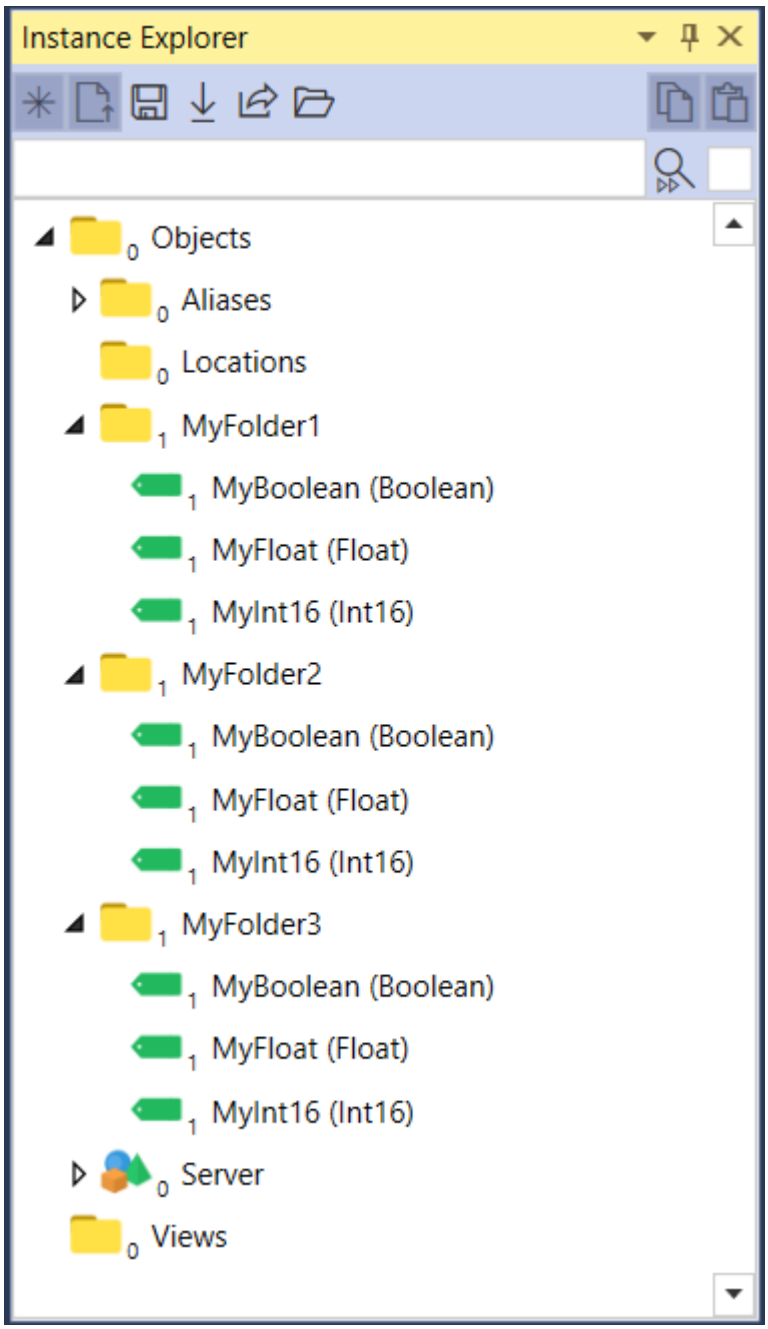


In the next step we want to add some variables to the folders.

Creating new variable or object instances is usually done by a drag and drop operation from a data type on the Type Explorer. Select the first folder in the Instance Explorer and then navigate to the desired data type in the Type Explorer. Drag and drop the data type onto the selected folder in the Instance Explorer. As a result, a corresponding instance of this data type is created there. In our example, we use the Boolean data type.



We now repeat this step with two more data types, e.g. Int16 and Float, and perform this process for the other two folders as well. If necessary, you can adjust the name of the added variable by double-clicking on it and then renaming it in the Attribute Explorer.

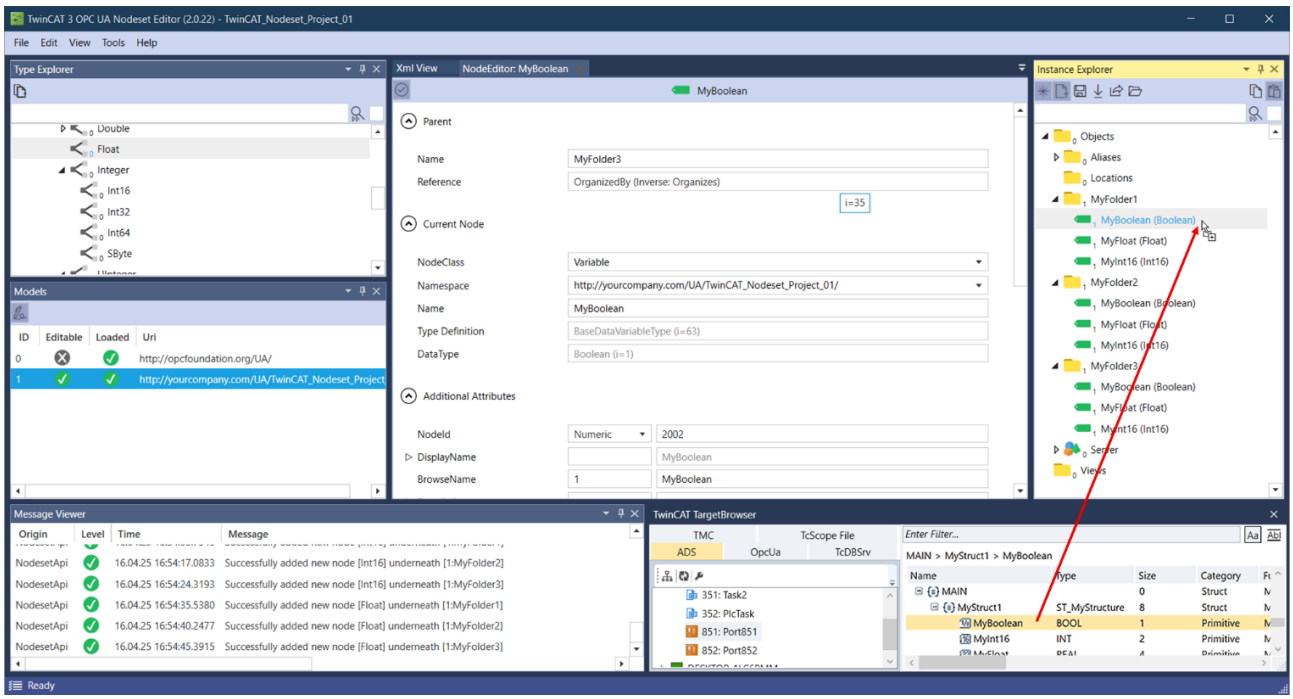


Linking with PLC variables

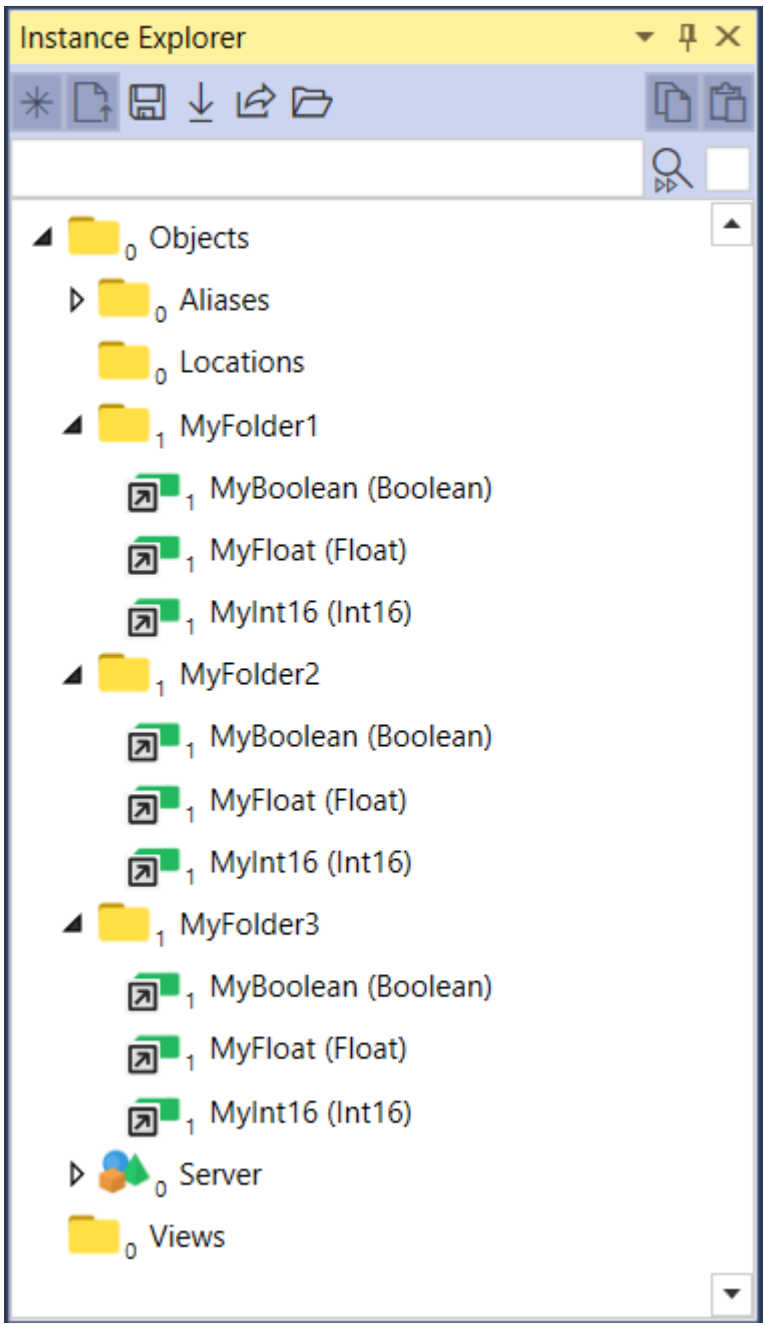
We want to link the now created instances with variables from our prepared PLC project. For this purpose, the TwinCAT Target Browser can be used, which you can open in the menu **Tools**.

Navigate to the prepared PLC project in the Target Browser and link the individual member variables of the ST_MyStructure data structure with the corresponding instances by dragging and dropping them onto the corresponding node in the Instance Explorer.

The link will then be visible in the tree structure and can be checked in the **NodeEditor**.

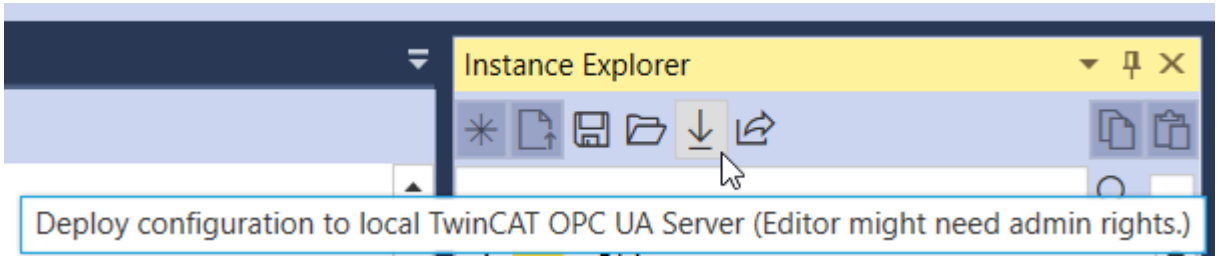


Perform this procedure for all variables.



Deployment of the project on the local TwinCAT OPC UA server

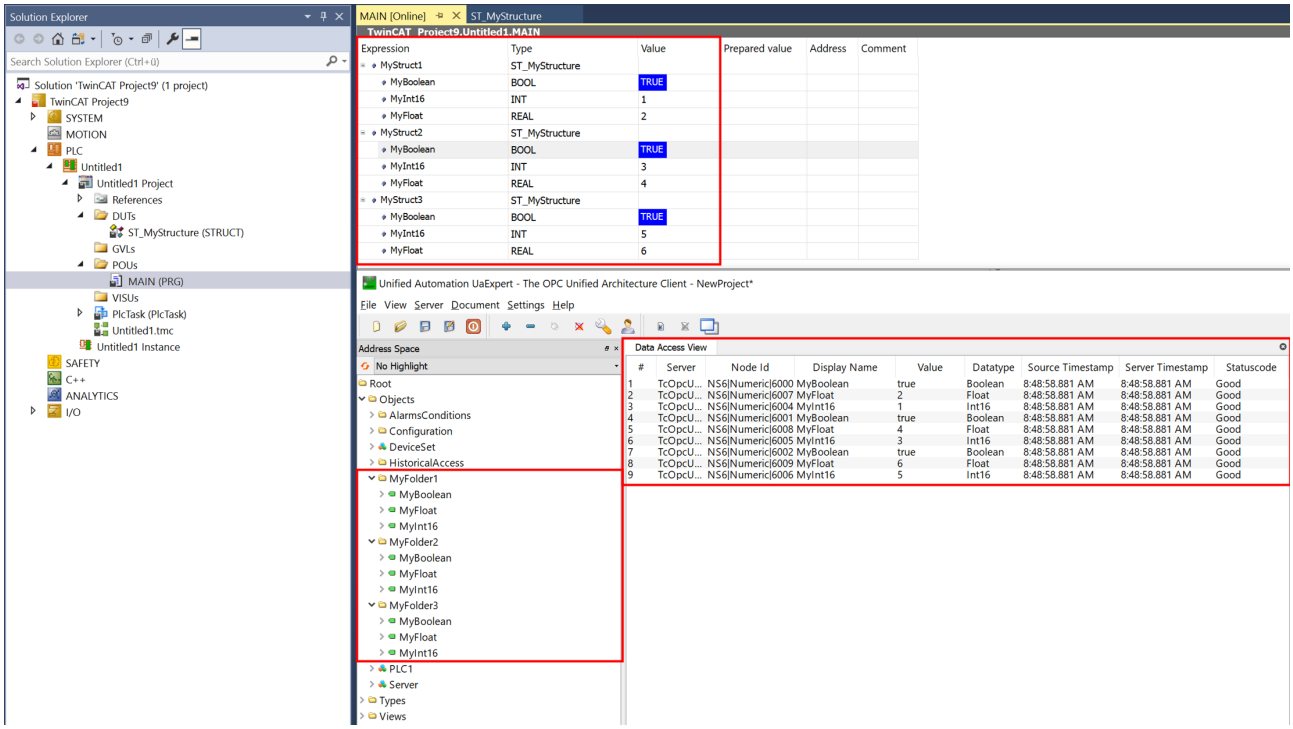
To deploy the project to the local TwinCAT OPC UA Server, click the **Deploy configuration to local TwinCAT OPC UA Server** button in the Instance Explorer toolbar.



This exports the project nodeset, including the necessary dependencies, to the locally installed TwinCAT OPC UA Server and also creates a corresponding configuration file for the server. The directory from the options or the project file is used as the target (see Application and project settings [▶ 43]).

Restart the TwinCAT OPC UA Server to load the configuration

The TwinCAT OPC UA Server must be restarted after the configuration has been adjusted in order to apply it. For this reason, the server should be restarted after deployment via the Nodeset Editor in order to load the new TcUaNodesetConfig and the nodesets configured in it. The address space of the server then contains all the nodes contained in the nodesets.



4.1.2 Simple companion specification

This documentation article is intended to get you started using this product quickly. After successful installation [▶ 10] and licensing [▶ 11], perform the following steps to create an initial nodeset based on a companion specification.

OPC UA Core Nodeset

By default, the TwinCAT OPC UA Nodeset Editor automatically loads the OPC UA Core Nodeset, which defines the namespace http://opcfoundation.org/UA/.

TF6100 TC3 OPC UA Server

This product works together with the TwinCAT OPC UA Server to run a created Nodeset and display it on the server's address space. Please ensure that the TF6100 TC3 OPC UA Server product is installed locally and/or on your target system.

Companion specification sample nodeset

As an example of a companion specification, a nodeset is used that defines an ObjectType "MyObjectType" with a few member variables. This nodeset can be downloaded from our TE6100 Samples repository on GitHub [▶ 58].

The following Quick Start tutorial will guide you through the following steps:

- Preparing a PLC project
- Creating a new project
- Importing the companion specification nodeset
- Creating object instances
- Linking the created instances with PLC variables
- Deploying the project to TwinCAT OPC UA Server

Preparing a PLC project

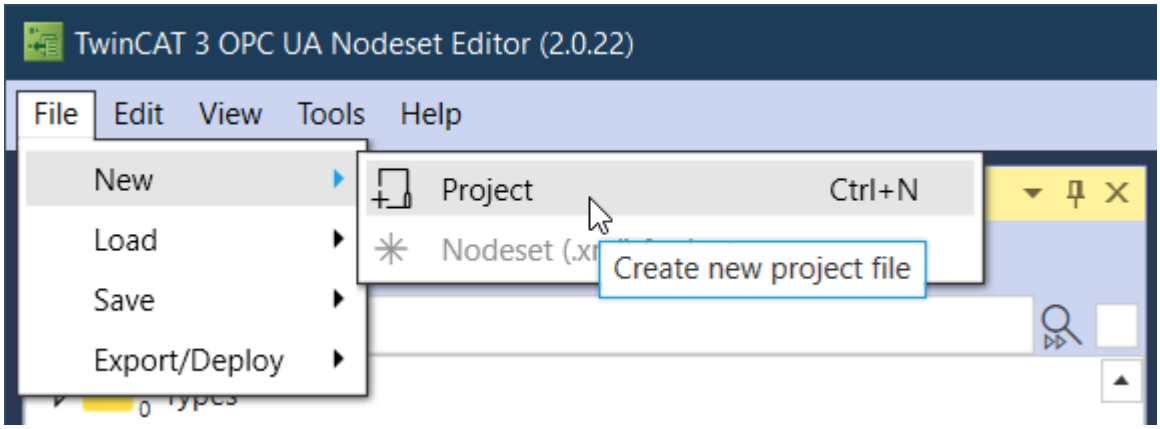
Create a new TwinCAT project and add a PLC project. Create the following data structure in the PLC project and instantiate it three times in the MAIN program.

```
TYPE ST_MyStructure :
STRUCT
  MyBoolean : BOOL;
  MyInt16 : INT;
  MyFloat : REAL;
END_STRUCT
END_TYPE

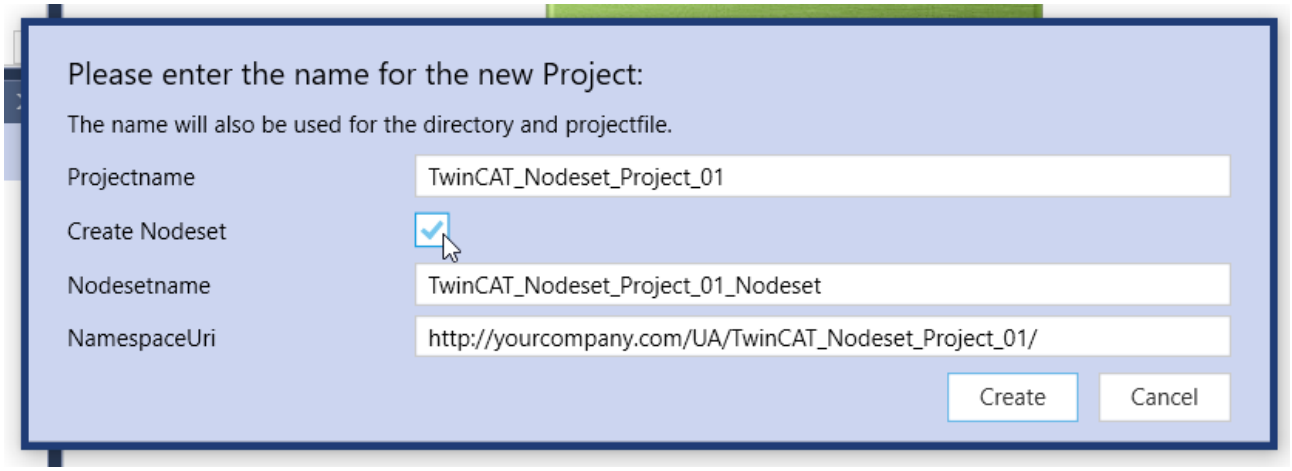
PROGRAM MAIN
VAR
  MyStruct1 : ST_MyStructure;
  MyStruct2 : ST_MyStructure;
  MyStruct3 : ST_MyStructure;
END_VAR
```

Creating a new project

To create a new Nodeset project, please open the menu **File** and click on **New > Project**.

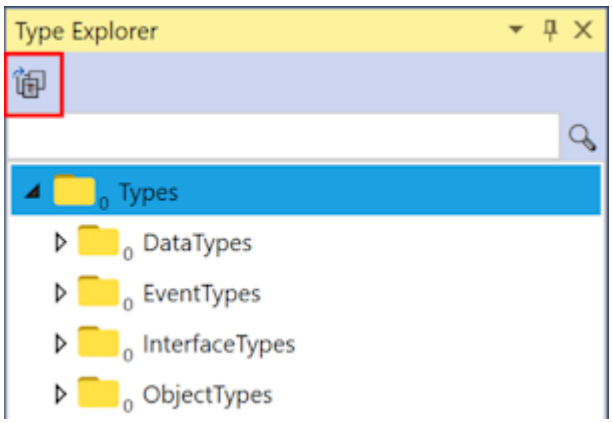


In the following selection dialog, you can configure the name for the project and, if the checkbox is activated (by default), a nodeset to be created. The name and the (namespace) Uri for a new information model can be defined in this nodeset.

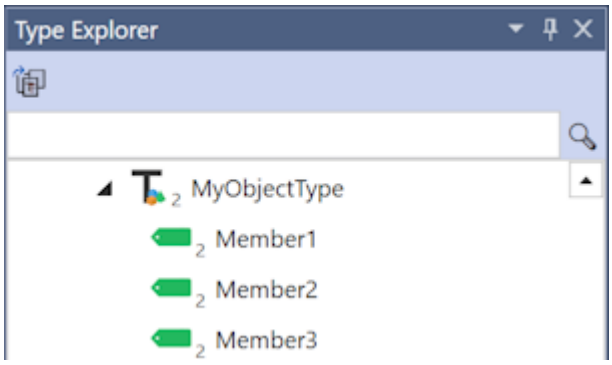


Importing the Companion Specification Nodeset

Import the file "te6100_quickstart_companion_specification.xml" from the GitHub repository into the TwinCAT OPC UA Nodeset Editor by clicking the button **LoadNodeset** in the Type Explorer.

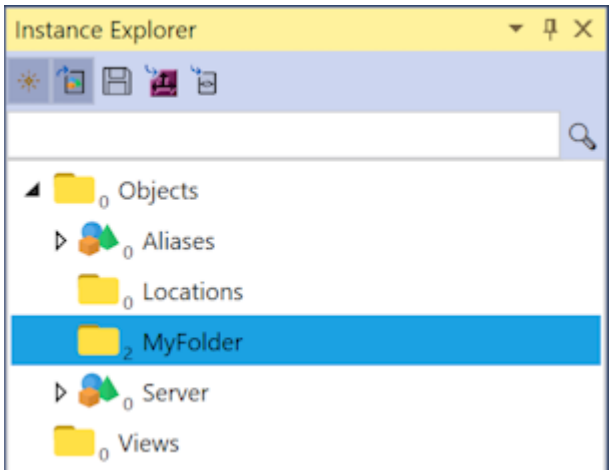


The import loads the type model from the nodeset into the Type Explorer and displays it there. The type model contains an ObjectType "MyObjectType", which we now want to instantiate. You can use the textbox to search for the type name or you can scroll to that type, which can be found under the ObjectTypes \ BaseObjectType node.



Creating object instances

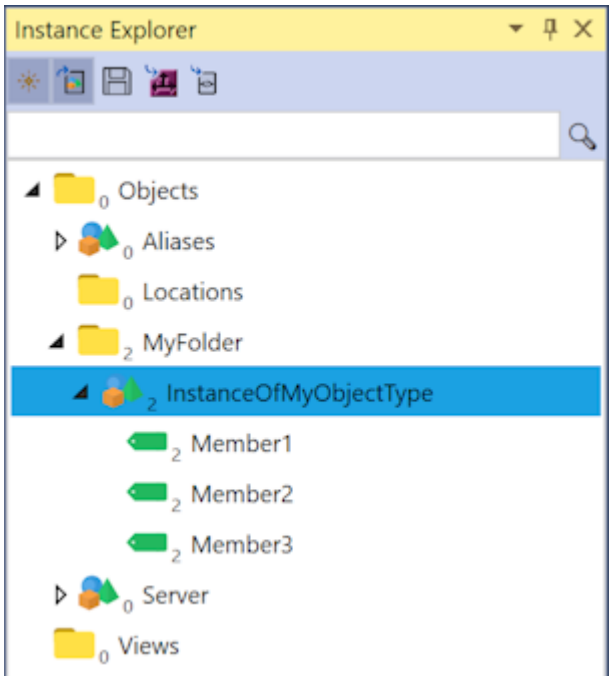
Use the context menu to create a new folder in the Instance Explorer. You can rename the folder by double-clicking on it and then renaming the "Name" attribute in the Attribute Explorer. Then save this change via the **Save** button in the Attribute Explorer toolbar.



Then drag and drop the type "MyObjectType" from the Type Explorer into the created folder in the Instance Explorer.



This creates a new object instance of the type "MyObjectType" and places it in the folder created. All mandatory member variables of the type are transferred accordingly to the Instance Explorer and displayed there as nodes below the object instance. You can now link these nodes with SPS variables.



You can rename the instance via the Attribute Editor.

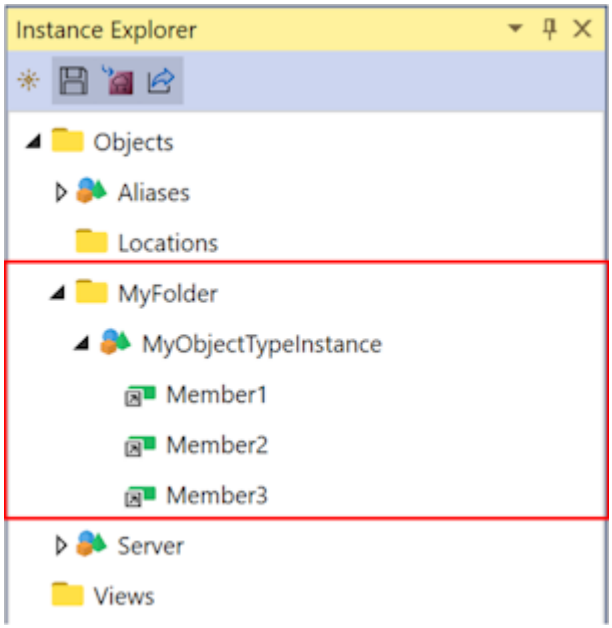
Linking the created instances with PLC variables

We now want to link the member variables of the now created object instance with variables from our prepared PLC project. For this purpose, the TwinCAT Target Browser can be used, which you can open in the menu **View**.

Navigate to the prepared PLC project in the Target Browser and link the individual member variables of the data structure with the corresponding member variables of the object instance by dragging and dropping them onto the corresponding node in the Instance Explorer. The data types here are as follows:

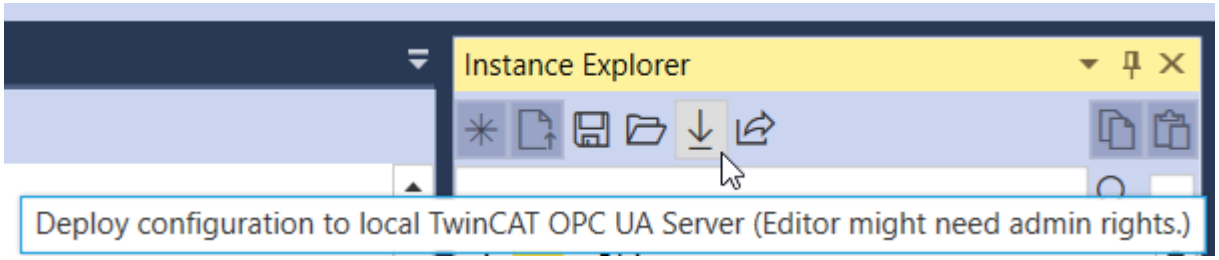
Member variable	Data type
Member1	Boolean (BOOL)
Member2	Int16 (INT)
Member3	Float (REAL)

You can also get this information from the Attribute Explorer if you double-click on the variable. As soon as you have linked all variables you can activate the project on the TwinCAT OPC UA Server in the next step.



Deployment of the project on the local TwinCAT OPC UA server

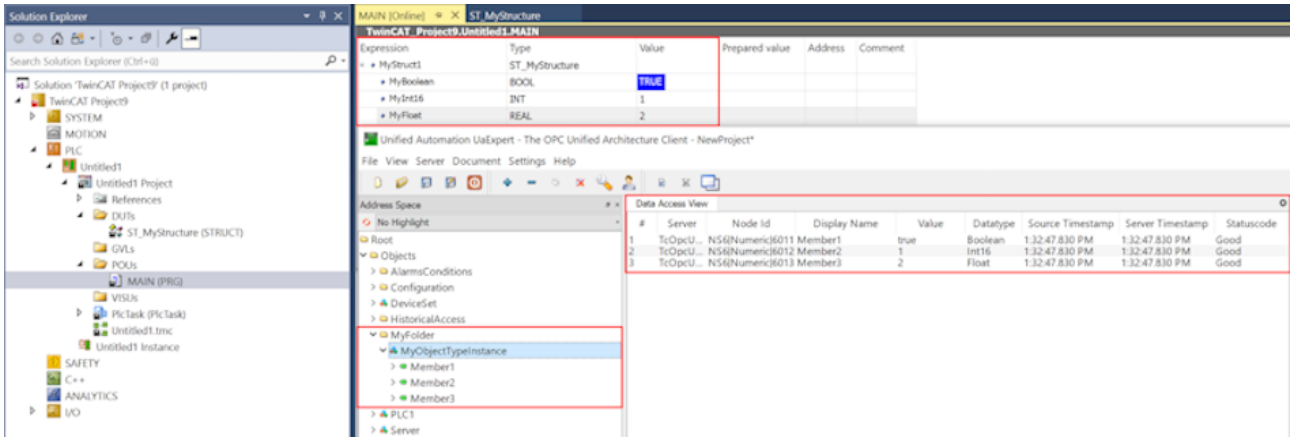
To deploy the project to the local TwinCAT OPC UA Server, click the **Deploy configuration to local TwinCAT OPC UA Server** button in the Instance Explorer toolbar.



This exports the project nodeset, including the necessary dependencies, to the locally installed TwinCAT OPC UA Server and also creates a corresponding configuration file for the server. The directory from the options or the project file is used as the target (see Application and project settings [▶ 43]).

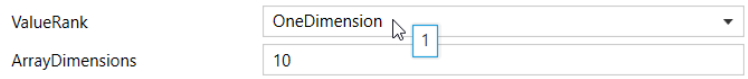
Restart the TwinCAT OPC UA Server to load the configuration

The TwinCAT OPC UA Server must be restarted after the configuration has been adjusted in order to apply it. For this reason, the server should be restarted after deployment via the Nodeset Editor in order to load the new TcUaNodesetConfig and the nodesets configured in it. The address space of the server then contains all the nodes contained in the nodesets.



4.2 Supported features

With the help of OPC UA, complex information models can be created. This page provides an overview about the currently available features of the TwinCAT OPC UA Nodeset Editor. Additional features will be provided as updates over time.

Feature	Description
Creation of nested folders	Allows the creation of (nested) folders, e.g. to structure instances.
Instantiation of data types	Permits the instantiation of data types from the OPC UA Core Nodeset. The following data types, (as well as their derived types), are supported: Boolean, SByte, Byte, Int16, UInt16, Int32, UInt32, Int64, UInt64, Float, Double, String, DateTime, ByteString, NodeId, LocalizedText, EUInformation, Range, TimeZoneDataType, UtcTime, OptionSet
Instantiation of data types as arrays	Enables the instantiation of arrays of data types from the OPC UA Base Nodeset. The above types are supported. To do this, the ValueRank and ArrayDimensions attributes must be set accordingly on the instance created: Example: For a one-dimensional array with a length of 10: 
Instantiation of object types (with and without methods)	Allows instantiation of object types from an imported nodeset, e.g. a companion specification. Certain restrictions may apply, see below.
Instantiation of structured data types	Enables the instantiation of structured types (all data types derived from “Structure”) from an imported nodeset, e.g. a companion specification. Certain restrictions may apply, see below. This may require further configuration steps, which are explained in more detail in our documentation about Mapping [▶ 35].
Mapping of variables and methods	Enables the linking of the created instances with elements from the PLC. The TwinCAT Target Browser (Tools => TargetBrowser) serves as the basis for this feature.
Online and offline mapping to various target systems	Mappings can be added to various active target systems via the **ADS** tab in the TargetBrowser. Mappings can be added offline using a TMC file via the **TMC** tab.
Handling of the ModelingRules (Optional, Mandatory, etc.)	When an object type is instantiated, sub-elements with the “Optional” and “Mandatory” ModelingRule are automatically added. Unwanted optional elements can be removed manually.
Export to local TwinCAT OPC UA Server	Enables the export of the created nodeset project to the locally installed TwinCAT OPC UA Server. This is explained in more detail under Exporting a project [▶ 46].
Export to a folder	Enables the export of the created nodeset project to a user-defined folder for manual copying to the server directory of a TwinCAT OPC UA Server. This is explained in more detail under Exporting a project [▶ 46].

Restrictions

Below you will find an overview of which features are currently not available, yet. Additional features are added over time via software updates of this product.

Feature	Description
Dynamic array lengths	The instantiation of arrays with dynamic length is possible, but such arrays cannot currently be used in the TwinCAT PLC or the TwinCAT OPC UA Server. This applies for ArrayDimensions = 0 or ValueRank = [-3 | -2 | 0].
Handling of the ModelingRules (OptionalPlaceholder, MandatoryPlaceholder, etc.)	Nodes with the "OptionalPlaceholder" and "MandatoryPlaceholder" ModelingRule must be created manually. This is currently only possible to a limited extent. Placeholder instances in the type cannot be instantiated directly. When instantiating manually under the instance of the correct data type, the reference type for the parent node cannot be modified (see below).
Create/Modify types	Creating new data or object types or changing existing types is currently not supported.
Static node values	It is currently possible to set a static value for a node using an experimental editor (this is only recommended for experts). However, static values in the Nodeset are not loaded by the TwinCAT OPC UA Server.
References	Currently the references of a node cannot be modified. When adding folders or Instances of nodes, these are referenced using **Organizes** references.
Mapping of structures with PLC keyword/ identifier conflicts	If an OPC UA structure type has a field with the name "Type", for example, this structure cannot currently be created in the PLC, as "Type" may not be used as the identifier of a PLC variable. This applies to all keywords. The following general rules for identifier assignment apply.

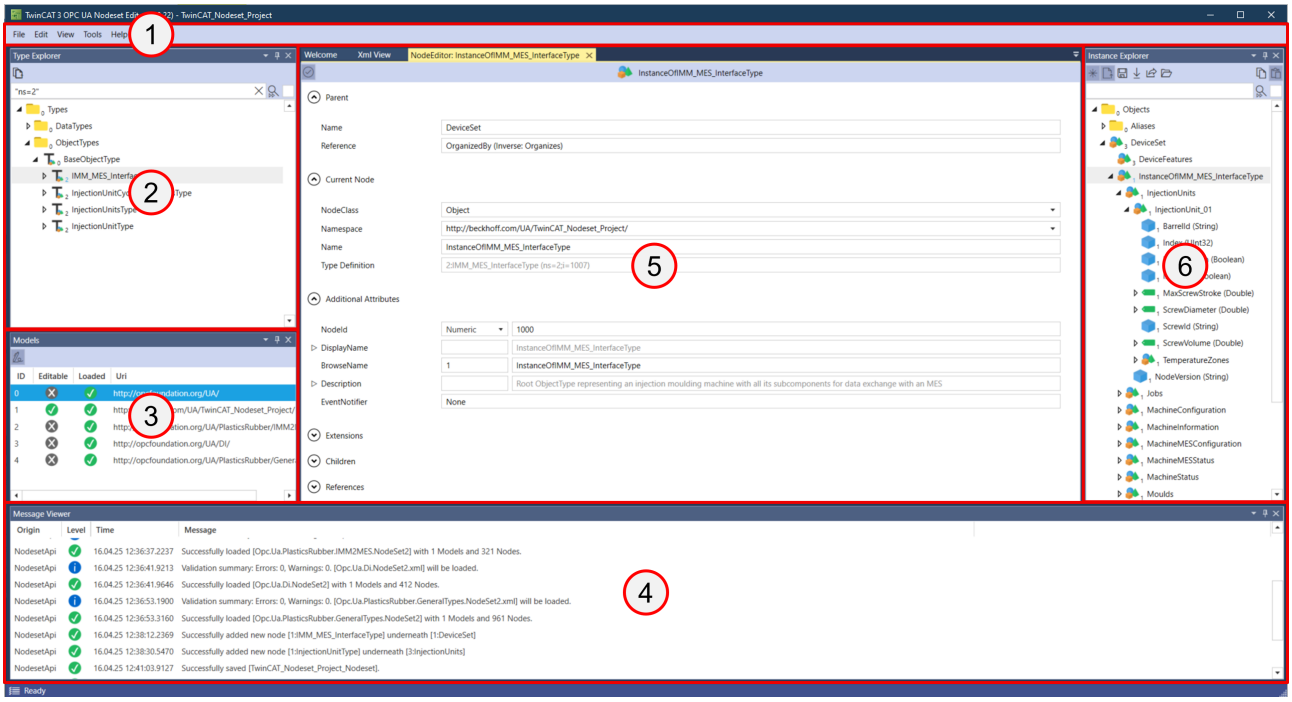
4.3 License information

There are different licenses that can be purchased for the TwinCAT OPC UA Nodeset Editor. The following table provides an overview of the different licenses that are available for this product.

License	Description
7-day trial license	A 7-day trial license can be used with this product. It allows all the functionalities of the TE6100 license (see below) but doesn't allow to save or deploy the project.
TE6100	This is the base license that is required to use the product. It allows access to all product functionalities but the ones defined by an add-on license.
TE6101	This license is an optional add-on license that allows to use the automatic code generation feature.

4.4 User interface

The user interface is divided into different areas, which will be described in more detail below. The following diagram illustrates the different areas.

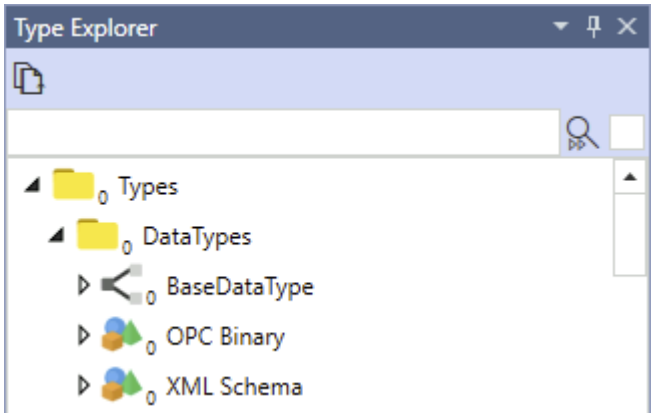


Area	Name	Description
1	Menu [▶ 30]	Access to Load/Save mechanisms, as well as settings for the user interface and the TwinCAT Target Browser.
2	Type Explorer [▶ 27]	Displays the type system of the currently loaded nodesets. A text box allows to search for nodes.
3	Models [▶ 29]	Displays the namespaces of the currently loaded nodesets.
4	Message Viewer [▶ 31]	Contains logging information about the currently loaded project.
5	Node Editor [▶ 29]	Displays the attributes of the currently selected node. The attributes of the nodes from both the Type Explorer and the Instance Explorer can be displayed here.
6	Instance Explorer [▶ 28]	Displays the currently configured instances. A text box allows to search for nodes.

The individual areas are arranged in a so-called docked window concept and can be arranged differently depending on user requirements. The windows of the Type, Attribute and Instance Explorer also contain their own toolbar, which can be used to perform actions related to this area. These actions are described in more detail in the corresponding article.

4.4.1 Type Explorer

The Type Explorer displays all data types from the currently loaded nodesets. Further actions are available via the toolbar and a text box allows to search for nodes.



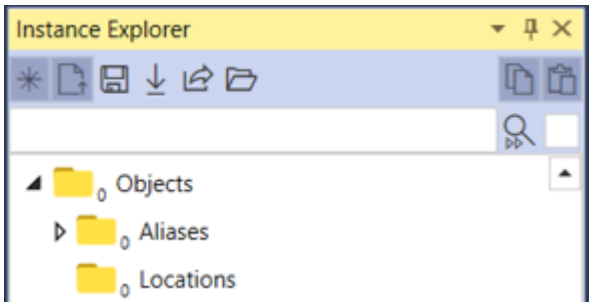
The numbers on each node represent the NamespaceIndex of a particular node and may be useful for easier identification of nodes.

The following table shows all actions that you can perform within the Type Explorer.

Action	Description
Import Nodeset	Allows to add more nodesets.
Drag&Drop	By using the Drag&Drop operation on a particular node and drag that node to the Instance Explorer, you can create an instance from that type.
Search	You can search for nodes by entering the name or NodeId of the node into the corresponding text box. Additional search strings are: ns=x (searches for nodes in a particular NamespaceIndex)

4.4.2 Instance Explorer

The Instance Explorer displays all instances in this project. Further actions are available via the toolbar.



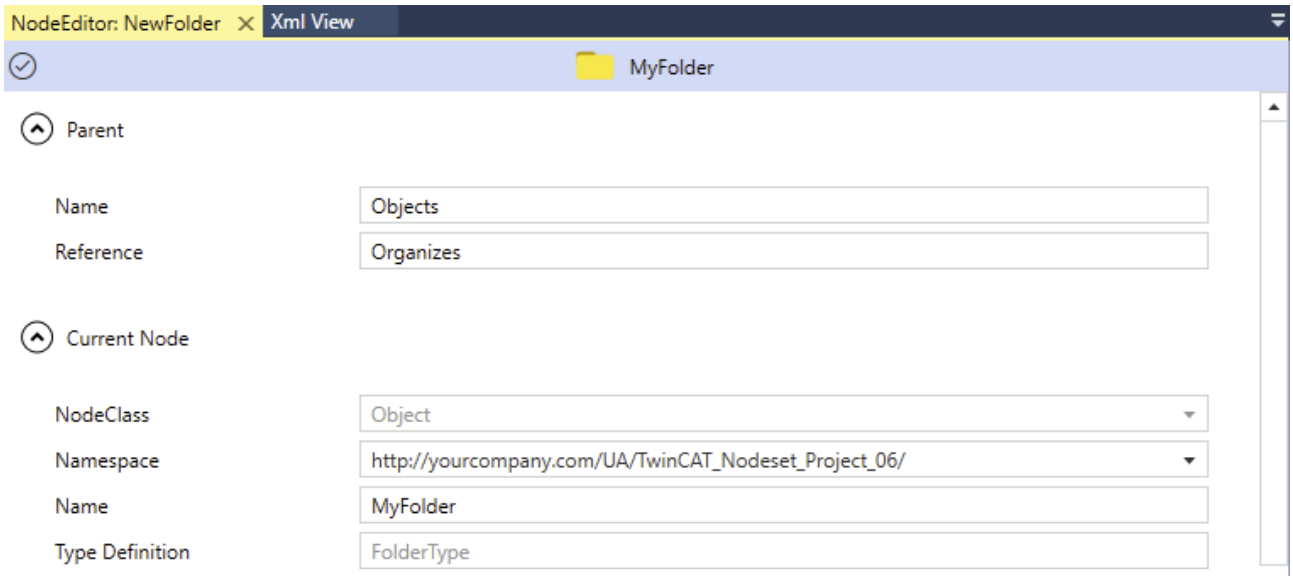
The following table lists all the actions you can perform in the Instance Explorer, arranged in the order of the buttons.

Action	Description
Create new Nodeset	Creates a new instance nodeset and automatically marks it as editable.
Load the nodeset to be edited (CTRL + ALT + O)	Loads an existing nodeset and automatically marks it as editable.
Save changes to the nodeset (CTRL + S)	Saves current changes to hard disk.
Deployment to the local TwinCAT OPC UA Server (CTRL + D)	Exports the current project to the local TwinCAT OPC UA Server.
Export nodesets (CTRL + E)	Exports the current project to a user-defined folder for manual copying to the server directory of a TwinCAT OPC UA Server. (The content is identical to Deploy)
Open project directory	Opens an Explorer window in the project directory.
Copy an instance (CTRL + C)	Copies the type definition of a selected instance.
Paste an instance (CTRL + V)	Creates a new instance of the previously copied type definition under the selected node and avoids name conflicts.
Search (CTRL - F in Instance Explorer)	You can search for nodes by entering the name or NodeId of the node into the corresponding text box. Additional search strings are: ns=x (searches for nodes in a particular NamespaceIndex) HasExtension (searches for nodes with a mapping to a PLC variable)

Action	Description
	HasNoExtension (searches for nodes without a mapping to a PLC variable) Quotation marks around the search parameter allow you to exclude overlapping hits. (Example: "ns=1" does not search for ns=10.)
Context menu (right mouse button) Delete node (DEL)	Removes the currently selected node and the closely connected subnodes. Please note that other subnodes (without the ParentNodeId attribute) are not removed and are considered to be "orphaned" if they have no other parent node.
Context menu (right mouse button) Remove mapping (SHIFT + DEL)	Removes the mapping of the selected node to a PLC variable, if available.
Context menu (right mouse button) Create folder (CTRL + SHIFT + N)	Creates a new instance of the type FolderType.

4.4.3 Node Editor

The Attribute Explorer displays the attributes of the currently selected node. Both the attributes of nodes from the Type Explorer and the Instance Explorer can be displayed. By double-clicking on a node, its attributes are loaded and displayed accordingly. Further actions are available via the toolbar.

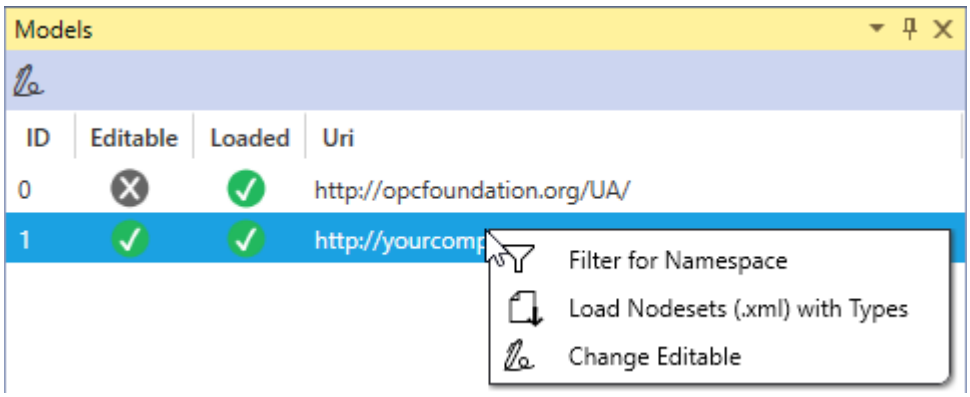


The following table shows all actions that you can perform within the Attribute Editor.

Action	Description
Edit attributes	Edit attributes of the currently opened node.
Save	Applies any changes to the attributes.
XML view	Allows to display the currently selected node as XML.

4.4.4 Models

The **Information Models** window shows all currently loaded namespaces – both from all loaded type or instance nodesets. This window allows you to choose which namespace you want to edit when creating instances out of a data or object type. Only one namespace can be edited at a time. The namespace that is currently marked as editable is used when creating new instances.



Nodesets can also be loaded via the context menu. A namespace can be selected as editable and filtered according to a namespace. A namespace can only be marked as editable if no other namespace is currently editable. If a namespace is to be marked as no more editable but the option is grayed out, unsaved changes must still be saved. If the editable nodeset is filtered, this is done in the Instance Explorer. If you filter for a non-editable nodeset, this is done in the Type Explorer. This feature uses the search in the Explorers, which you can customize manually.

4.4.5 Menu

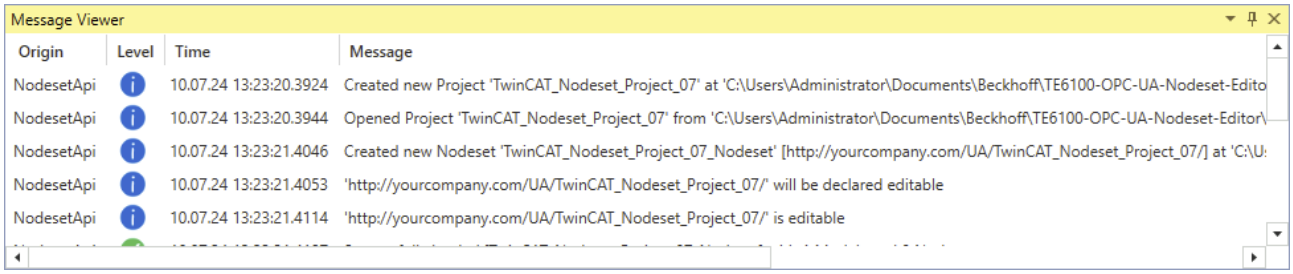
The menu includes different mechanisms related to the currently opened project. The following table provides more information about each menu entry.

Menu	Subitem	Subitem	Description
File	New	Project	Creates a new project.
File	New	Nodeset (.xml) for Instances	Creates a new nodeset and marks it as editable.
File	Load	Project	Loads an existing project.
File	Load	Nodeset (.xml) with Types	Loads a choice of existing nodesets.
File	Load	Nodeset (.xml) with Instances	Loads an existing nodeset and marks it as editable.
File	Save	Nodeset (.xml) with Instances	Saves the editable nodeset.
File	Export/Deploy	TcUaNodesetConfig.xml + Nodeset (.xml) with Instances + Types	Exports the currently opened project to a user-defined folder.
File	Export/Deploy	TcUaNodesetConfig.xml + Nodeset (.xml) with Instances + Types	Exports the currently open project to the TwinCAT OPC UA Server.
Edit	Undo		Undoes an action in a selected field.
Edit	Redo		Repeats an action in a selected field.
Edit	Cut		Cuts out the selected text.
Edit	Copy		Copies the selected text.
Edit	Paste		Pastes previously copied content.
View	Type Explorer		Shows/Hides the Type Explorer.
View	Models		Shows/Hides the Models window.
View	Instance Explorer		Shows/Hides the Instance Explorer.
View	Message Viewer		Shows/Hides the Message Viewer.
View	XML view		Shows/hides the XML view.
View	Welcome		Shows/Hides the Welcome screen.
Tools	Target Browser		Starts the TwinCAT Target Browser or brings it to the front.
Tools	Nodeset Converter		Starts the converter dialog for converting old projects (version < 2.0.0).
Tools	Options		Opens the settings window.

Menu	Subitem	Subitem	Description
Help	Info		Displays product and license information.
Help	OPC UA | Nodeset Repository		Opens the GitHub page for UaNodesets of the OpcFoundation.
Help	OPC UA | Online Reference		Opens the official documentation of the OpcFoundation
Help	TE6100 | Online Documentation		Opens the product documentation
Help	TE6100 | Product Website		Opens the product website
Help	TwinCAT 3 | Licensing		Opens the licensing overview page

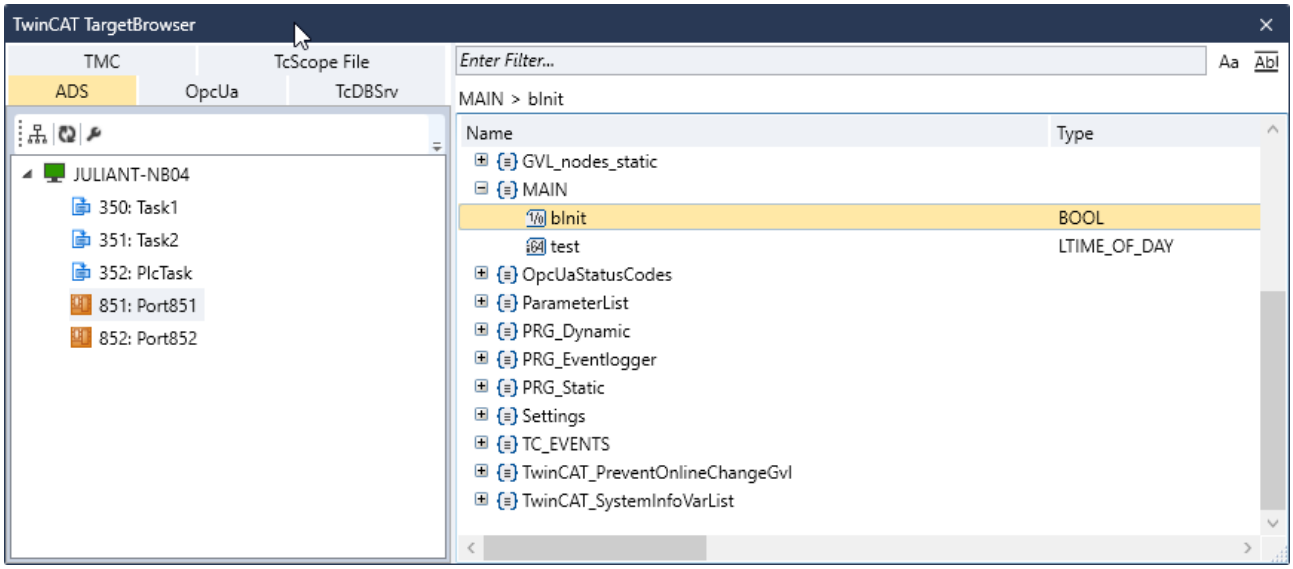
4.4.6 Message Viewer

The Message Viewer contains logging information from your currently opened project. Messages are categorized into different severity levels and their origin. In addition, each message has a timestamp that indicates when the message has been thrown.



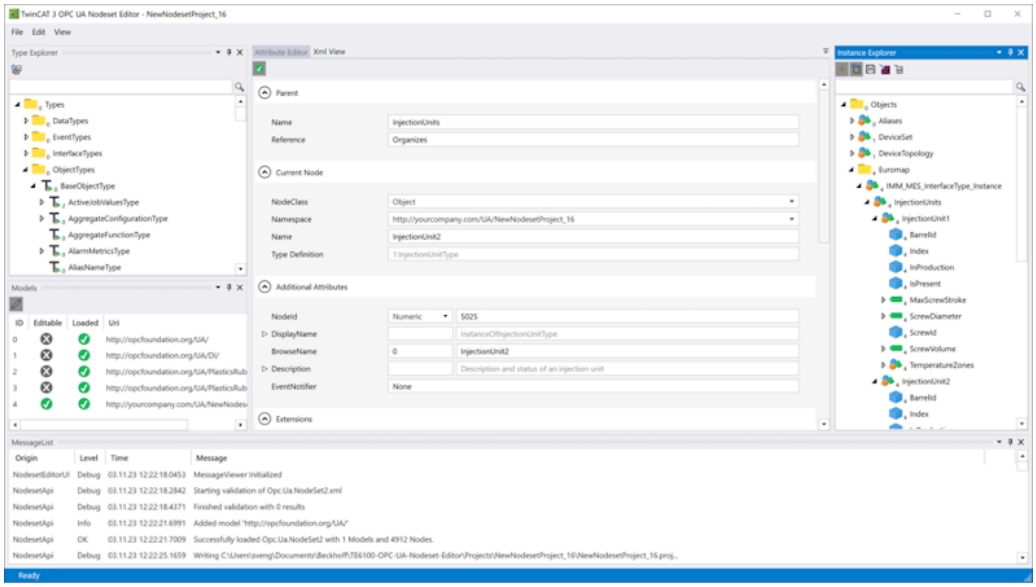
4.4.7 TwinCAT Target Browser

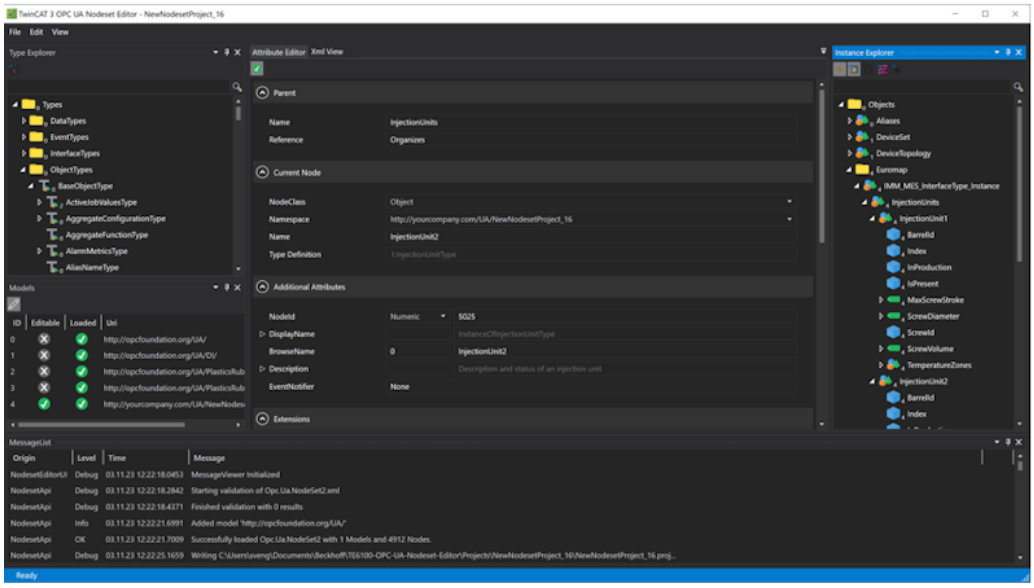
The TwinCAT Target Browser can be opened via the **View** menu. Currently, it is implemented as a separate window, but it will be integrated into the main window structure in future versions.

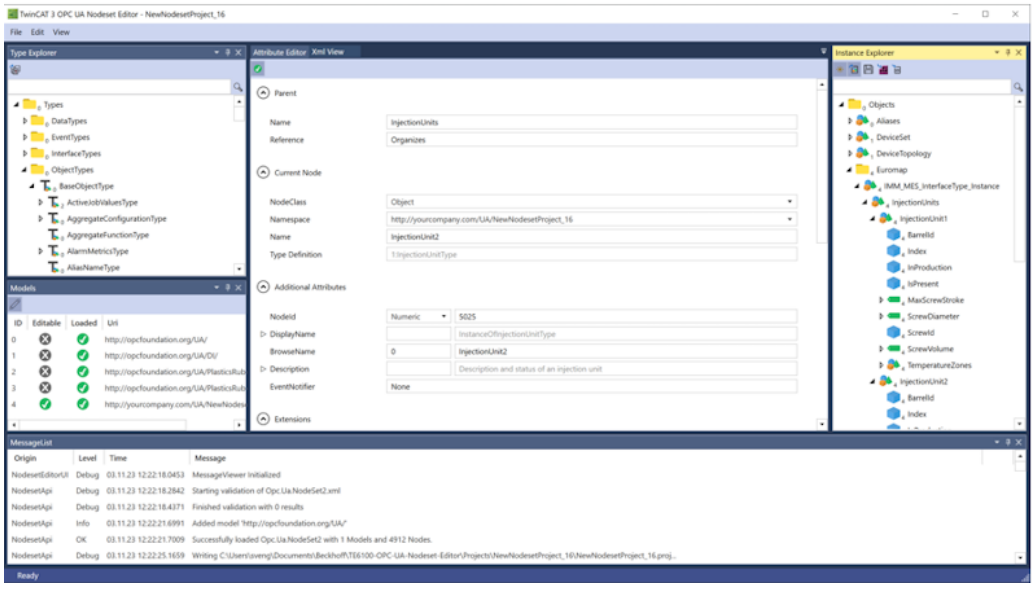


4.4.8 Themes

The TwinCAT OPC UA Nodeset Editor currently supports three different color themes: Light, Dark and Blue (default). You can select the theme of your choice via the **View** menu. The setting can be persisted in the application settings [▶ 43].







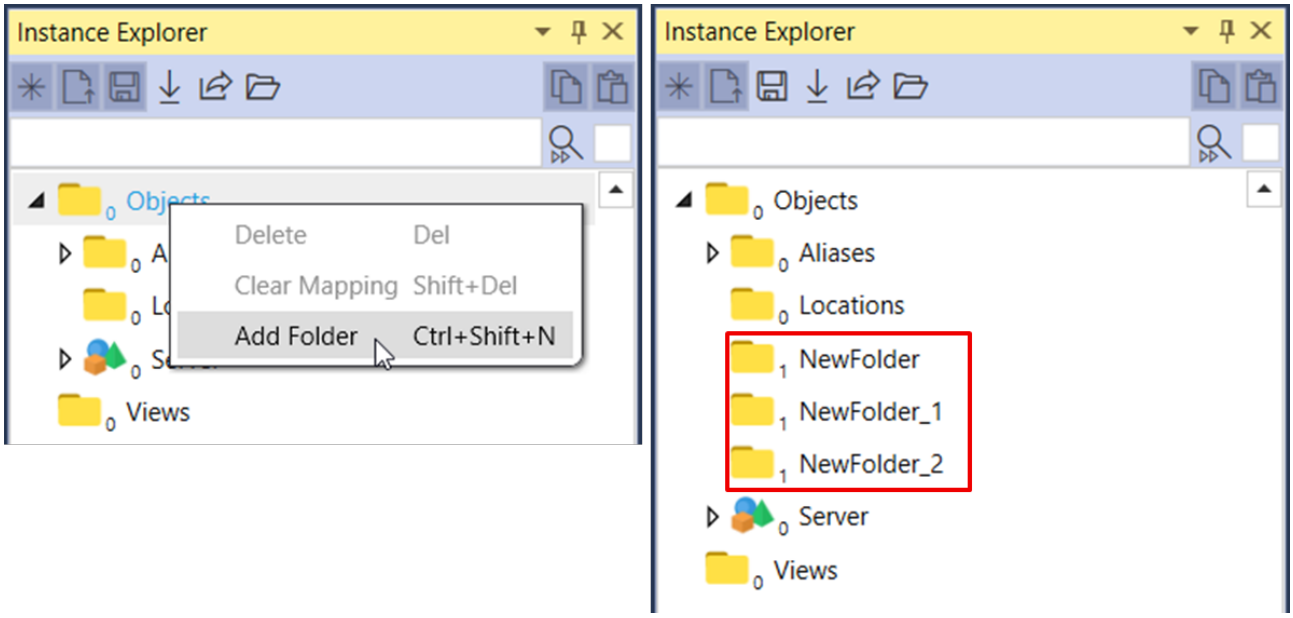
4.5 Modeling

4.5.1 Instantiation

The following node classes can be created in the Nodeset Editor: UA_Object, UA_Variable and UA_Method. The editor supports the instantiation of UA_DataType nodes and UA_ObjectType nodes for this purpose. Typical instantiation scenarios are described in more detail below:

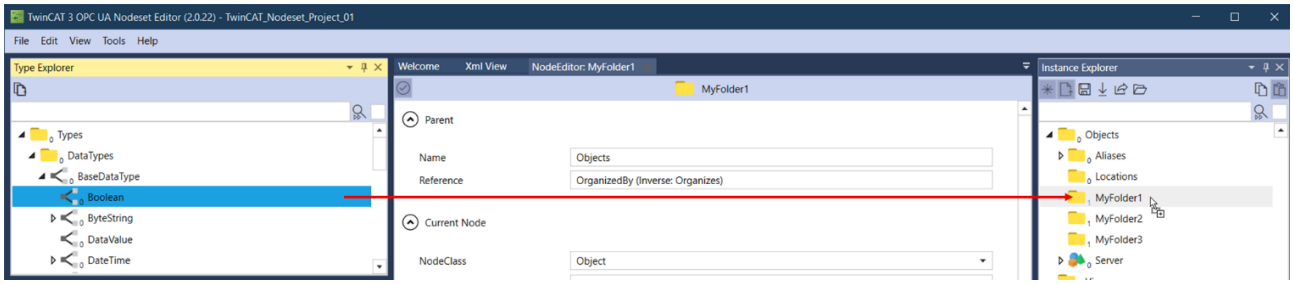
Create a folder:

As instances of the ObjectType "FolderType" are often used to structure the namespace, it is possible to add this directly under the clicked node via the context menu. Alternatively, the specified keyboard shortcut can be used if a node is chosen in the Instance Explorer:



Creating a variable or an object

Creating new variable or object instances is usually done by a drag and drop operation from a data type or object type on the Type Explorer. Navigate to the desired data type in the Type Explorer and drag and drop it onto a node in the Instance Explorer. As a result, a corresponding instance of this data type is created there. In our example, we use the Boolean data type.



UA_ObjectType nodes with their entire hierarchy can also be created as instances according to the same scheme.

4.5.2 Arrays

In OPC UA, arrays are described for UA_Variable nodes using the ValueRank and ArrayDimensions attributes. The actual DataType always specifies the element type, for example Int32, Double or String.

ValueRank

ValueRank fixes whether a variable is a scalar or an array and how many dimensions the array has.

- -3: Scalar or one-dimensional array
- -2: any, i.e. scalar or array
- -1: Scalar, i.e. not an array
- 0: Array with one or more dimensions
- 1: one-dimensional array
- n: n-dimensional array

ArrayDimensions

ArrayDimensions describes the size of the individual dimensions.

The value is specified as a list of dimensions. The number of entries in ArrayDimensions should match the number of dimensions in ValueRank. If the size is not to be fixed, ArrayDimensions can be left empty or 0.

Examples:

- 0: n-dimensional array with any number of elements
- 10: one-dimensional array with 10 elements
- 3,4: two-dimensional array with 3 × 4 elements
- 2,5,8: three-dimensional array with 2 × 5 × 8 elements

The values can be edited in the Node Editor [▶ 29] using the following elements. While the ValueRank only allows permissible values, the value itself must be checked for the ArrayDimensions:



Examples ValueRank and ArrayDimensions

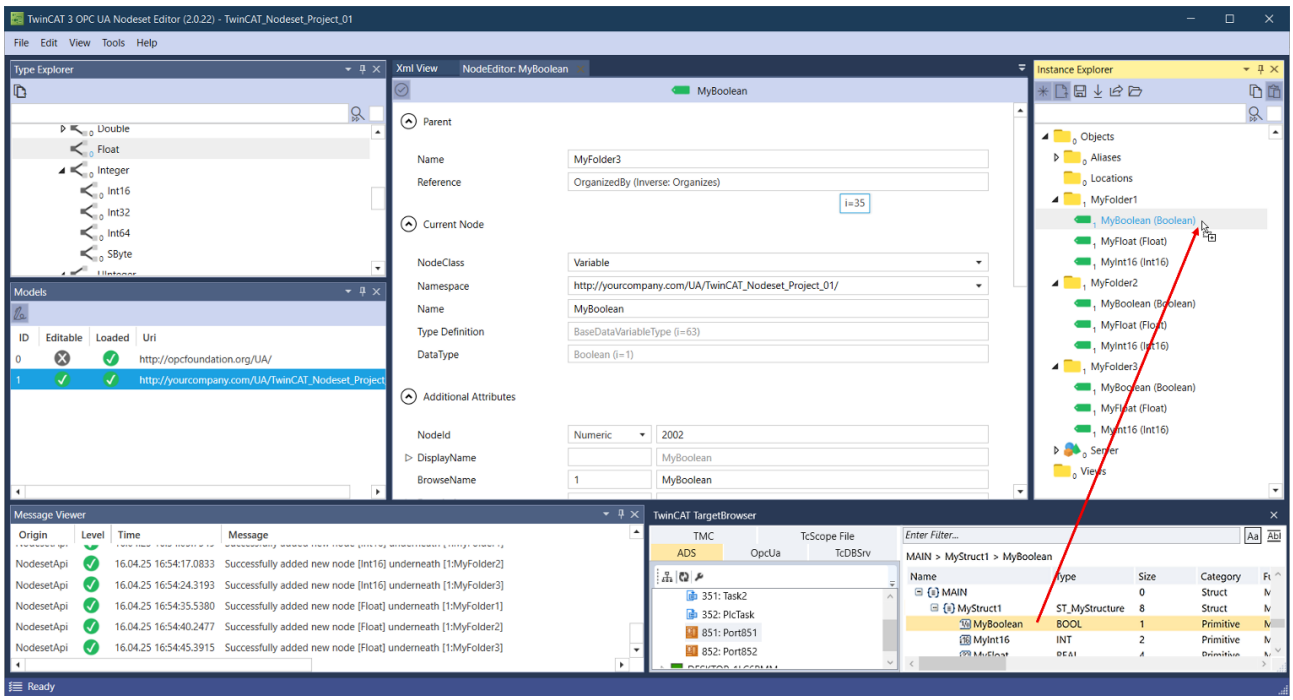
ValueRank = -1 → Scalar, ArrayDimensions remains empty

ValueRank = 1 → one-dimensional array, e.g. ArrayDimensions = 10

ValueRank = 2 → two-dimensional array, e.g. ArrayDimensions = 3,4

4.6 Mapping

The TwinCAT Target Browser is available for linking OPC UA nodes to PLC symbols, which will add the address information of a PLC symbol to an OPC UA node. The nodeset can then be made available on the TwinCAT OPC UA Server. The mapping basically works by dragging a symbol from the target browser onto an OPC UA instance (drag and drop):



Objects

Objects cannot be linked to PLC symbols since they do not have a value that could be linked.

Variables

Variables (and properties) can be linked to PLC variables using the TwinCAT Target Browser.

Requirements

1. The data type of the PLC variables corresponds to the data type of the OPC UA node.
   Depending on the setting (Tools -> Options -> Advanced -> Allow mapping despite validation errors & Disable warning dialog), you will be prevented from mapping or notified if a mapping does not fit (perfectly).

Further information regarding the assignment can be found at Data type assignment TwinCAT <=> OPC UA [▶ 40].

Structures

Structures can be linked to a PLC structure using the TwinCAT Target Browser.

Requirements

1. The data structure in the PLC matches the OPC UA data structure. In particular:
   - Sequence of fields
   - Data types
2. The PLC type definition contains the OPC UA type information as attributes.
   The TwinCAT OPC UA Server requires these attributes in order to assign the PLC data type to the correct OPC UA data type. Simple PLC data types do not require these attributes.

The required attributes can be written manually above the instance or type declaration or retrieved for copying via the TwinCAT OPC UA Nodeset Editor.

Step-by-step with the Nodeset Editor:

1 - Finding and opening the structure type.

5 - Creating the DUT in TwinCAT.

2 – Selection and copying of all names and DataType BrowseNames of the fields

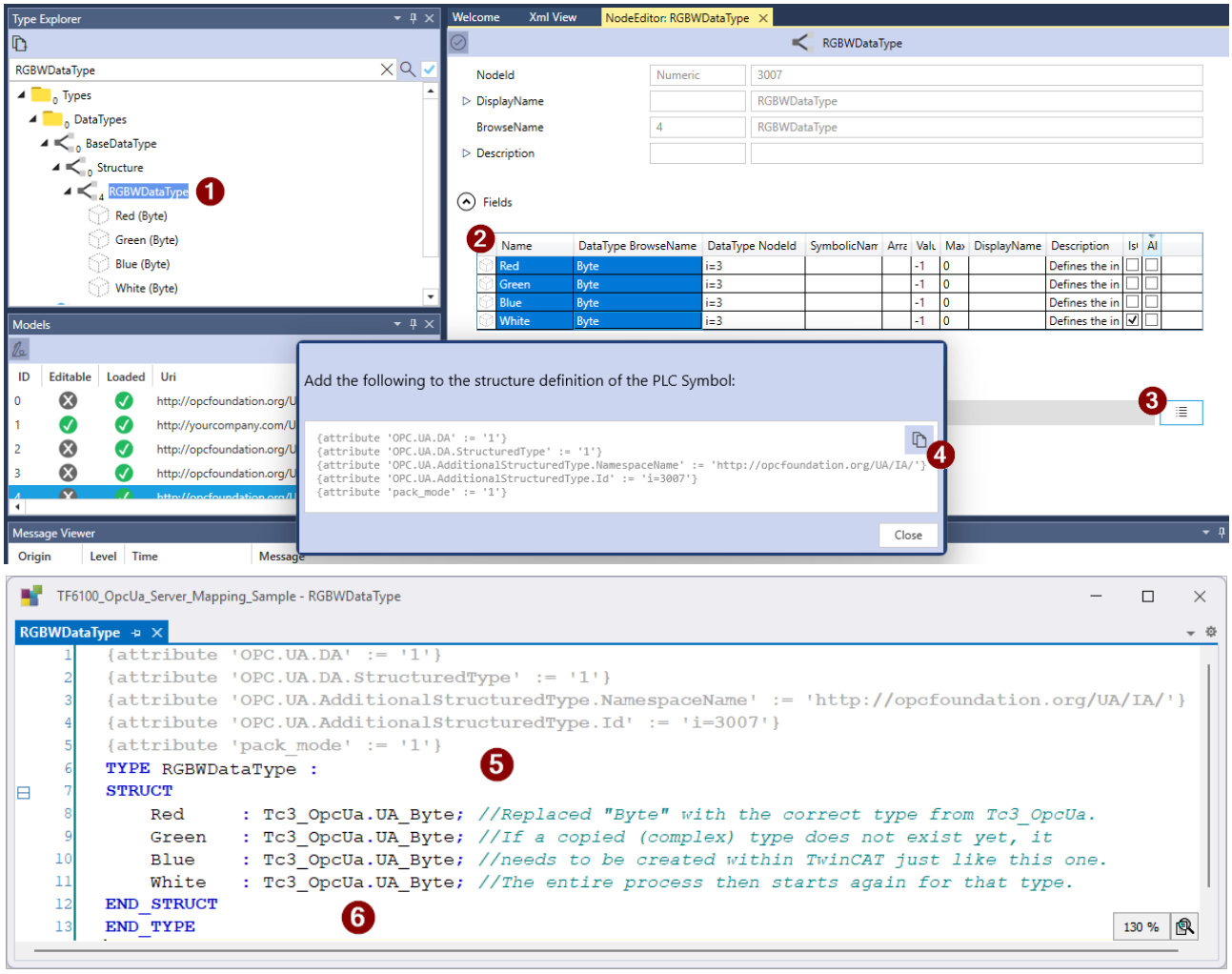
5 - Insert in TwinCAT between STRUCT and END_STRUCT

3 - Opening of the pragmas for the type

4 - Copying the pragmas

5 - Inserting in TwinCAT above TYPE

6 - Replacing the DataType BrowseNames with the appropriate PLC types (e.g. Boolean => "Tc3_OpcUa.UA_Boolean" or "BOOL").
Missing (structure) types may also need to be created.



Meaning of the attributes:

- **OPC.UA.DA**
  Activates the provision of the data type and any sub-elements via OPC UA.
- **OPC.UA.DA.StructuredType**
  Identifies the type as a structured OPC UA data type.
- **OPC.UA.AdditionalStructuredType.NamespaceName**
  Specifies the NamespaceUri of the data type from the nodeset.
- **OPC.UA.AdditionalStructuredType.Id**
  Contains the NodeId of the OPC UA data type, in this case i=3007.
- **pack_mode := '1'**
  Ensures that the memory layout of the structure is defined and compatible.

As soon as an instance of the PLC structure type has been declared, it can be linked in the same way as a variable.

Methods

Methods can be linked to a PLC function block via the TwinCAT Target Browser.

Requirements

1. The PLC function block has the structure of a so-called job method. This is explained in Job methods in the TwinCAT OPC UA Server documentation.
2. The input and output parameters of the PLC function block correspond to the OPC UA method.

Example

As the example OPC UA method is extensively defined, only the key data of the OPC UA method is given here:

Four InputParameters: Boolean, Int16, String, Enum (INT16)

Four OutputParameters: Boolean, Int16, String, Enum (INT16)

The PLC function block looks like this:

```
//Funktionsbaustein Deklaration:

{attribute 'OPC.UA.DA.JobMethod' := 'Method2_WithSimpleArguments'}
FUNCTION_BLOCK PUBLIC FB_Method2_WithSimpleArguments
VAR
// Timer to simulate job execution
timer : TON := (PT:=T#4S);

// Variables to determine if a job has been started (Start() method has been called) or finished
jobRunning : BOOL;
jobFinished : BOOL;

// State machine variable
state : UDINT;

//Values of method specific input parameters:
bArgIn : BOOL;
nArgIn : INT;
sArgIn : STRING;
eArgIn : E_Enum_1;

//Values of method specific output parameters:
bArgOut : BOOL;
nArgOut : INT;
sArgOut : STRING;
eArgOut : E_Enum_1;

END_VAR

//Funktionsbaustein Implementierung
CASE state OF
0:
IF jobRunning THEN
state := state + 1;
END_IF

1: //Logic of method in here:
timer(IN:=TRUE);
IF timer.Q THEN

//Set Inputs To Outputs
bArgOut := bArgIn;
nArgOut := nArgIn;
sArgOut := sArgIn;
eArgOut := eArgIn;

timer(IN:=FALSE);
jobRunning := FALSE;
jobFinished := TRUE;
state := 0;
END_IF

END_CASE
```

As required by every job method, the function block has three PLC methods:

```
//Start Deklaration:

{attribute 'TcRpcEnable' := '1'}
METHOD PUBLIC Start : HRESULT
VAR_INPUT
SimpleArgIn1 : BOOL;
SimpleArgIn2 : INT;
SimpleArgIn3 : STRING;
SimpleArgIn4 : E_Enum_1;
END_VAR
VAR_OUTPUT
hdl : UDINT; // handle, can be used for concurrent calls
END_VAR

//Start Implementierung:
IF jobRunning THEN
Start := Tc3_Module.E_FAIL;
ELSE
//Give inputs to functionblock body
bArgIn := SimpleArgIn1;
nArgIn := SimpleArgIn2;
sArgIn := SimpleArgIn3;
eArgIn := SimpleArgIn4;

hdl := 16#BE6780FF; //static because concurrent calls are disabled by checking jobRunning
jobRunning := TRUE;
Start := OpcUaStatusCodes.Good; //Set return value of method
END_IF

//CheckState Deklaration:
{attribute 'TcRpcEnable' := '1'}
METHOD PUBLIC CheckState : HRESULT
VAR_INPUT
hdl : UDINT; // handle, can be used for concurrent calls
END_VAR
VAR_OUTPUT
bBusy : BOOL; // Do not change. Used by server to find out if the job is finished

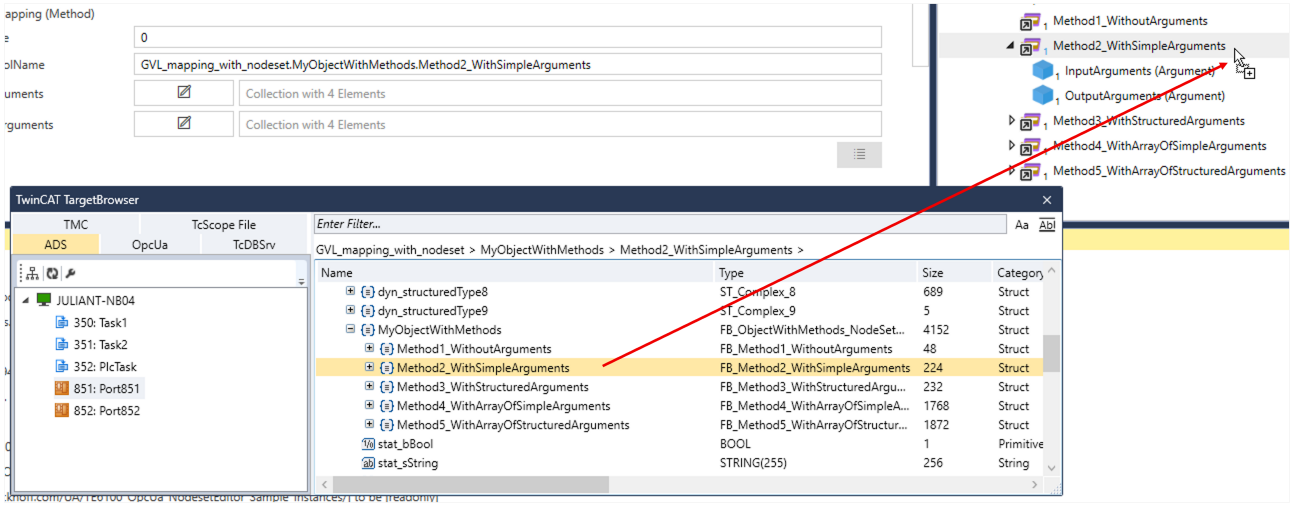
SimpleArgOut1 : BOOL;
SimpleArgOut2 : INT;
SimpleArgOut3 : STRING;
SimpleArgOut4 : E_Enum_1;
END_VAR

//CheckState Implementierung:
IF hdl <> 16#BE6780FF THEN
CheckState := OpcUaStatusCodes.BadInternalError; //Set return value of method
ELSIF NOT jobFinished THEN
bBusy := TRUE; // job is not finished yet
CheckState := OpcUaStatusCodes.Good; // done, no error occured during execution (OPC UA StatusCode
"GOOD")
ELSE
//Give values to outputs
SimpleArgOut1 := bArgOut;
SimpleArgOut2 := nArgOut;
SimpleArgOut3 := sArgOut;
SimpleArgOut4 := eArgOut;

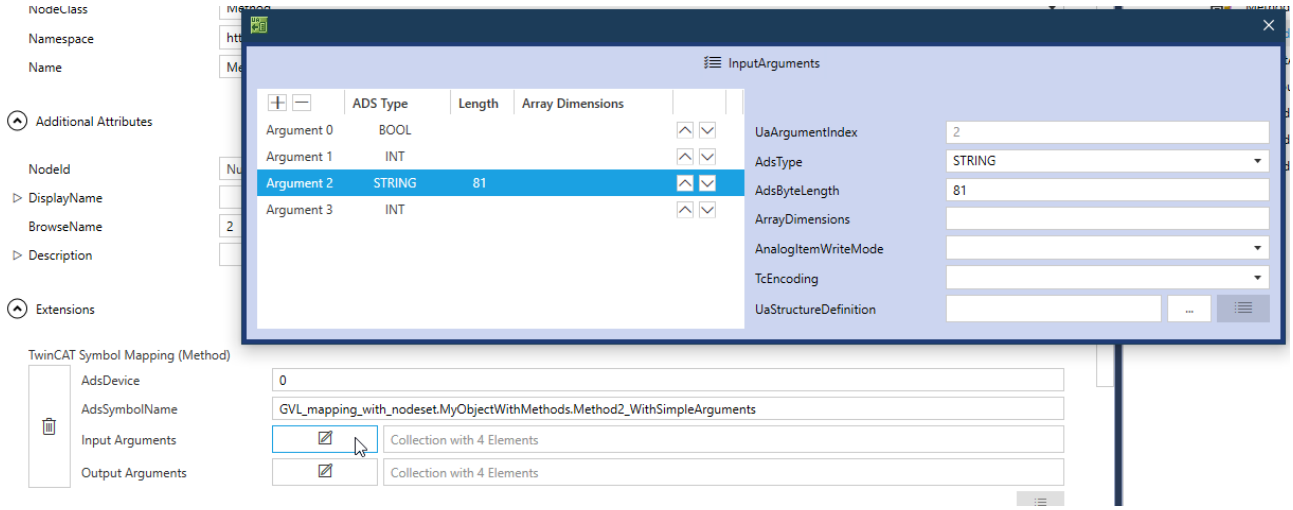
bBusy := FALSE; // job is finished
CheckState := OpcUaStatusCodes.Good; // done, no error occured during execution (OPC UA StatusCode
"GOOD")
END_IF

//Abort Deklaration:
{attribute 'TcRpcEnable' := '1'}
METHOD PUBLIC Abort : HRESULT
VAR_INPUT
hdl : UDINT; // handle, can be used for concurrent calls
END_VAR
```

As soon as an instance of the PLC function block has been created, it can be mapped in the same way as a variable. After the mapping…



...the parameters can be checked via the TwinCAT Symbol Mapping using the following buttons and adjusted if necessary. OPC UA and PLC must be compatible! There is (still) no validation of method links.



4.6.1 Data type assignment TwinCAT <=> OPC UA

The following three tables provide an overview of the assignment of TwinCAT data types (IEC61131-3) and their corresponding OPC UA data types (Built-In types) as defined in the OPC UA Core Namespace http://opcfoundation.org/UA/.

TwinCAT 3 library - Tc3_OpcUa

When installing the editor via TcPkg, the Tc3_OpcUa library is installed. This contains some type definitions that can be used in combination with the editor.

The following table assigns each TwinCAT data type to its OPC UA BaseDataType (Built-In) according to the PLC-Open specification. Please note that there is no OPC UA data type for the TwinCAT data type BIT. In addition, the TwinCAT special data types are treated separately.

TwinCAT data type	OPC UA BaseDataType (Built-In)
__XINT *(x64: LINT | x86: DINT)*	(see below for platform specific type)
__UXINT *(x64: ULINT | x86: UDINT)*	(see below for platform specific type)
__XWORD *(x64: LWORD | x86: DWORD)*	(see below for platform specific type)
BIT	-
BOOL	Boolean
SINT	SByte
USINT	Byte
INT	Int16

TwinCAT data type	OPC UA BaseDataType (Built-In)
UINT	UInt16
DINT	Int32
UDINT	UInt32
LINT	Int64
ULINT	UInt64
BYTE	Byte
WORD	UInt16
DWORD	UInt32
LWORD	UInt64
REAL	Float
LREAL	Double
TIME	Int64
LTIME	Int64
DATE	DateTime
LDATE	Int64
TIME_OF_DAY / TOD	UInt32
LTIME_OF_DAY / LTOD	Int64
DATE_AND_TIME / DT	DateTime
LDATE_AND_TIME / LDT	Int64
STRING	String
WSTRING	String

The following table assigns the possible TwinCAT data types to each OPC UA BaseDataType (Built-In). Please note that there is no TwinCAT data type (yet) for some OPC UA data types.

OPC UA BaseDataType (Built-In)	TwinCAT data type
Boolean	BOOL
SByte	SINT
Byte	USINT, BYTE
Int16	INT, WORD
UInt16	UINT, WORD
Int32	DINT, DWORD
UInt32	UDINT, DWORD, TOD
Int64	LINT, LTIME, LDATE, LTOD, LDT
UInt64	ULINT, LWORD
Float	REAL
Double	LREAL
String	STRING, WSTRING
DateTime	DATE, DT
Guid	-
ByteString	*Tc3_OpcUa*.UA_ByteString
XmlElement	-
NodeId	*Tc3_OpcUa*.UA_NodeId
ExpandedNodeId	-
StatusCode	UDINT
QualifiedName	-
LocalizedText	*Tc3_OpcUa*.UA_LocalizedText
ExtensionObject => (i=22 | Structure)	Abstract type. See below for a selection of concrete examples.
DataValue	-

OPC UA BaseDataType (Built-In)	TwinCAT data type
Variant => (i=24 | BaseDataType)	Abstract type.
DiagnosticInfo	-

The following table assigns the appropriate TwinCAT data types to each OPC UA BaseDataType (not Built-In) from the OPC UA Core specification.

OPC UA BaseDataType (not Built-In)	TwinCAT data type
EUInformation *[ExtensionObject]*	*Tc3_OpcUa*.UA_EUInformation
Range *[ExtensionObject]*	*Tc3_OpcUa*.UA_Range
TimeZoneDataType *[ExtensionObject]*	*Tc3_OpcUa*.UA_TimeZoneDataType
UtcTime *[DateTime]*	*Tc3_OpcUa*.UA_UtcTime
OptionSet *[ExtensionObject]*	*Tc3_OpcUa*.UA_OptionSet & *Tc3_OpcUa*.UA_OptionSetOption

4.7 Application directories

This application uses different directories to store relevant information, e.g. configuration files, nodeset libraries and files related to the current project.

Installation folder

The installation folder of this application is located under the TwinCAT installation directory.

```
%TcInstallDir%\Functions\TE6100-OPC-UA-Nodeset-Editor
```

Configuration files

This application uses different configuration files, which are located in a subfolder of the ProgramData directory:

```
%ProgramData%\Beckhoff\TE6100-OPC-UA-Nodeset-Editor
```

Libraries

There is a central directory in which you can store nodeset files that you repeatedly need to use in your projects as libraries. Each time you want to import a nodeset file into your current project, the TwinCAT OPC UA Nodeset Editor opens this “Library” directory, which is located in a subfolder of the user's home directory:

```
%UserProfile%\Documents\Beckhoff\TE6100-OPC-UA-Nodeset-Editor\
```

Projects

Per default, new projects are created in a subfolder of the user’s home directory:

```
%UserProfile%\Documents\Beckhoff\TE6100-OPC-UA-Nodeset-Editor\Projects
```

Log files

The TwinCAT OPC UA Nodeset Editor creates log files that may contain useful information in case anything goes wrong. These log files are located in a subfolder of the ProgramData directory:

```
%ProgramData%\Beckhoff\TE6100-OPC-UA-Nodeset-Editor\Log
```

Deploy and export directory

The TwinCAT OPC UA Nodeset Editor can export the current project either to the locally installed TwinCAT OPC UA Server or to a user-defined directory. The directories can be set in the options and are as follows by default:

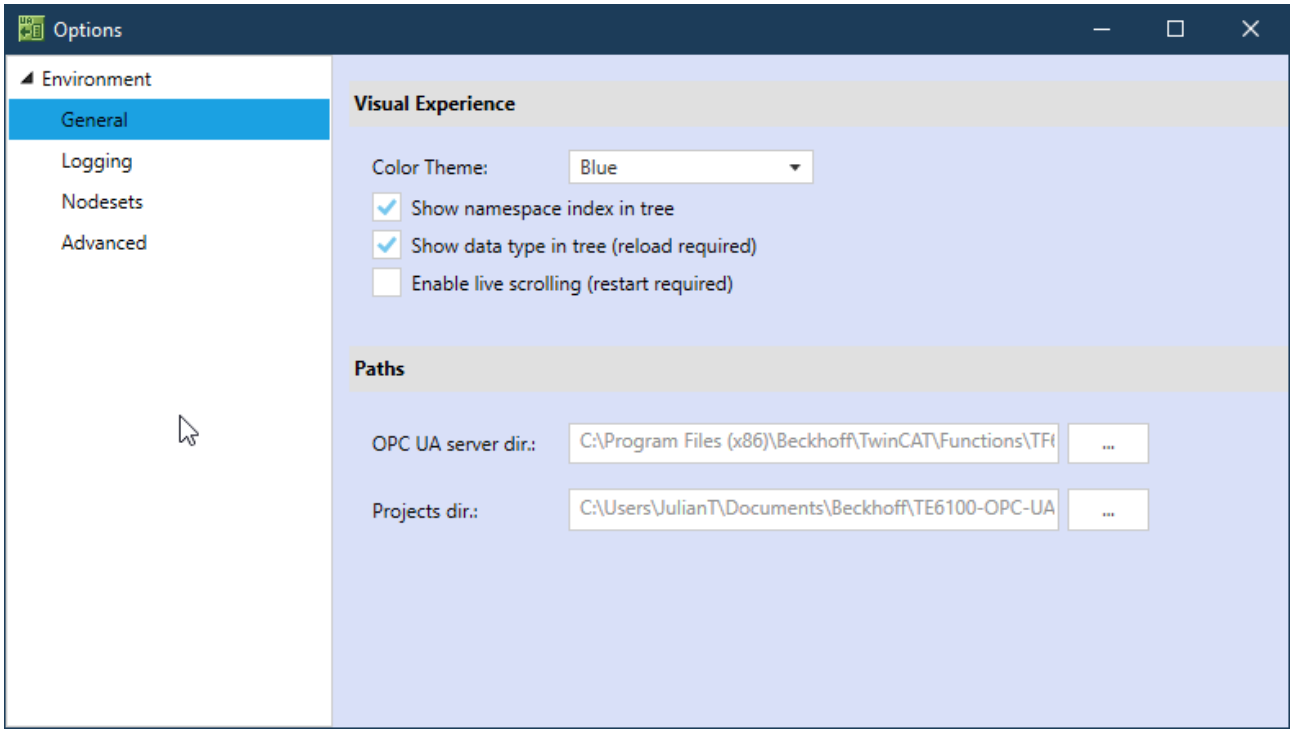
```
%TcInstallDir%\Functions\TF6100-OPC-UA\Win64\Server\
%UserProfile%\Documents\Beckhoff\TE6100-OPC-UA-Nodeset-Editor\Projects\<CurrentProject>\Export
```

4.8 Application and project settings

The application itself and each project includes XML-based configuration files, which contain different settings that influence the behavior of the application. Such settings are described in the following chapters.

Application settings

The configuration file directory (see Application directories [▶ 42]) includes the file TE6100.config, which contains configuration parameters for the TwinCAT OPC UA Nodeset Editor application. There is a graphical dialog for this file in the user interface under **Tools** => **Options**. Unless otherwise stated, changes apply immediately.



The following options are available.

Section	Option	Description	Possible values [default value]
General	Color Theme	Sets the default theme for the application.	[blue], dark, light
General	Show namespace index in tree	Enables/disables the display of the NamespaceIndex as a number in the Type and Instance Explorer.	[true], false
General	Show data type in tree	Enables/disables the display of the DataType in brackets in the Type and Instance Explorer for variables.	[true], false
General	Enable live scrolling	Enables that the view is updated live when the scrollbar is moved in the tree. Has a significant impact on performance when searching and scrolling in larger projects.	true, [false]
General	OPC UA server dir	Default storage location of the TwinCAT OPC UA Server installation. This path is used when deploying a nodeset to the TwinCAT OPC UA Server.	Selectable path, [%TcInstallDir% \Functions\TF6100-OPC UA\Win64\Server\]

Section	Option	Description	Possible values [default value]
General	Projects dir	Default base directory for new projects.	Selectable path, [%UserProfile% \Documents\Beckhoff\TE6100-OPC-UA-Nodeset-Editor]
Logging	Enable Logging	Enables/disables logging.	[true], false
Logging	Output Directory	Path to the log file	Selectable path, [%ProgramData% \Beckhoff\TE6100-OPC-UA-Nodeset-Editor\Log\]
Logging	Severity Level	Sets the severity level at which messages are logged and displayed in the Message Viewer. Debug activates all messages and is not recommended for productive use.	Debug, None, [Info], Warning, Error, OK, Fatal
Nodesets	Core Nodeset File Path	Path to the OPC UA base nodeset (used for all base data and object types). This nodeset is loaded by default for every new project.	Selectable path, [%UserProfile% \Documents\Beckhoff\TE6100-OPC-UA-Nodeset-Editor\Library\Opc.Ua.NodeSet2.xml]
Nodesets	Library Directory	Default base directory for type nodesets. This directory is opened by default whenever you import type nodesets.	Selectable path, [%UserProfile% \Documents\Beckhoff\TE6100-OPC-UA-Nodeset-Editor\Library\]
Nodesets	Order Attributes On Export	If this option is set to true, the attributes of all XML elements are sorted in alphabetical order, with common attributes at the top (e.g. NodeId and BrowseName) and others following in alphabetical order. Tip: If you want to compare a nodeset saved by this software with another nodeset (e.g., with “git diff”), it may be useful to set this to false to reduce the number of changes.	[true], false
Nodesets	Order Nodes On Export	If the setting is "true", the nodes are sorted uniformly according to NamespaceId and NodeId. Tip: If you want to compare a nodeset saved by this software with another nodeset (e.g., with “git diff”), it may be useful to set this to false to reduce the number of changes.	[true], false
Nodesets	Xml Schema File Path	Path to the OPC UA Nodeset XmlSchemaFile, which is used for the (syntax) validation of nodesets.	Selectable path, [%ProgramData% \Beckhoff\TE6100-OPC-UA-Nodeset-Editor\UANodeSet.xsd]
Advanced	AllowDownloadFrom Internet	The Nodeset Editor already contains a specific version of the base nodeset. By changing this setting to “true”, the current version will be downloaded from the OPC Foundation instead.	true, [false]

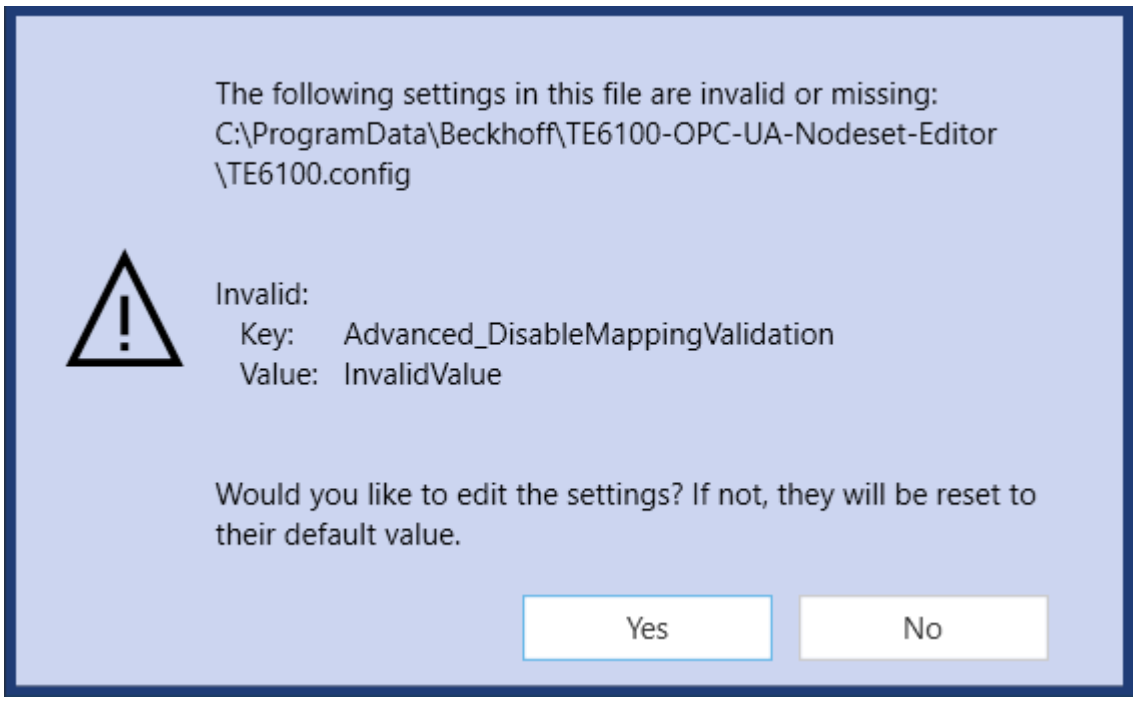
Section	Option	Description	Possible values [default value]
Advanced	Allow mapping despite validation errors	If activated, all mappings are allowed, regardless of whether the mapping validation fails. (Mapping a BOOL to an Int32, for example, would otherwise be prevented.)	[true], false
Advanced	Disable warning dialog	If activated, the message explaining why a mapping validation fails will no more be displayed.	true, [false]
Advanced	Enable value editor	Activates an experimental editor for editing the values of-N.	true, [false]
Advanced	Allow add of user specific extensions	Activates an experimental editor for editing user-defined extensions of instance nodes.	true, [false]
Advanced	Show new name prompt when instantiating new nodes	Activates a dialog that permits you to name an instance node directly during drag and drop.	true, [false]

Project settings

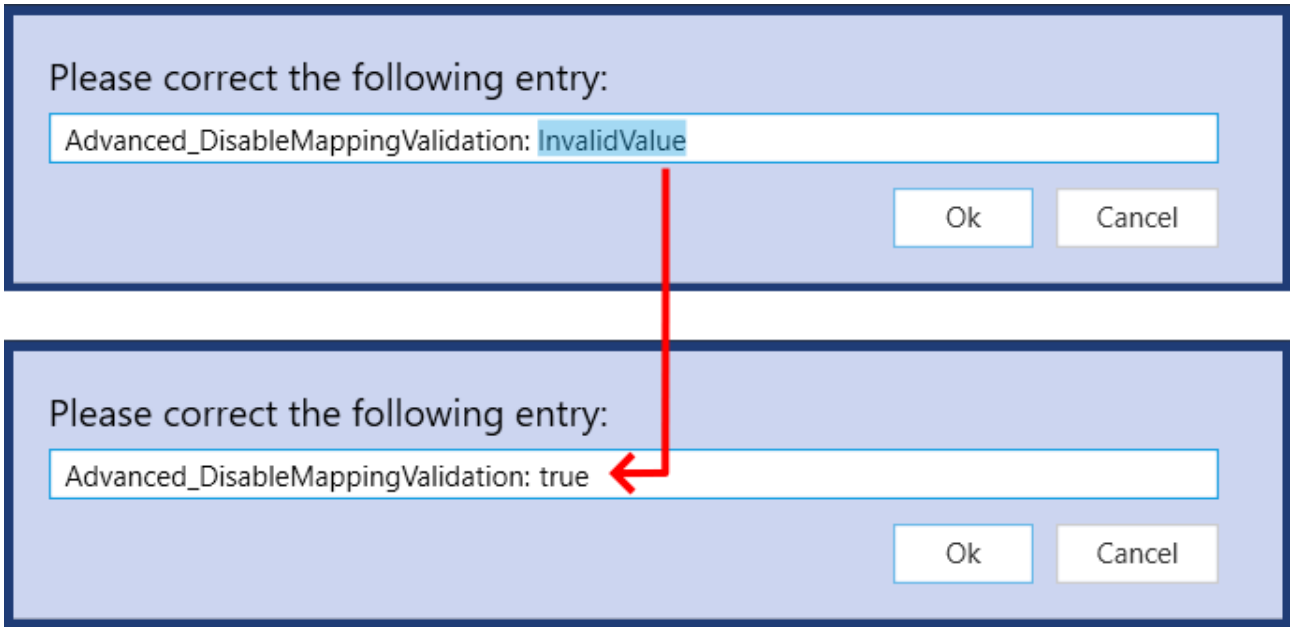
There is no graphical dialog for these settings yet. Changes to this file require the editor to be stopped beforehand or the project to be unloaded, as changes could otherwise be overwritten.

Parameter	Description
ExportDir	This directory is opened by default if the project is to be exported to a user-defined folder.
LocalOpcUaServer	This path is used when a project is exported to the local TwinCAT OPC UA Server. When creating a new project, this path corresponds to the default value (see above - General | OPC UA Server Dir), but you can override it for specific projects.
Nodesets	Contains a list of all nodesets that belong to this project including their path, NamespaceName, version and PublicationDate.

If an invalid configuration is recognized, you will be presented with a dialog informing you about the invalid parameter.



Declining the dialog will reset the value to its default setting. Alternatively, you can confirm the dialog and correct the value manually:



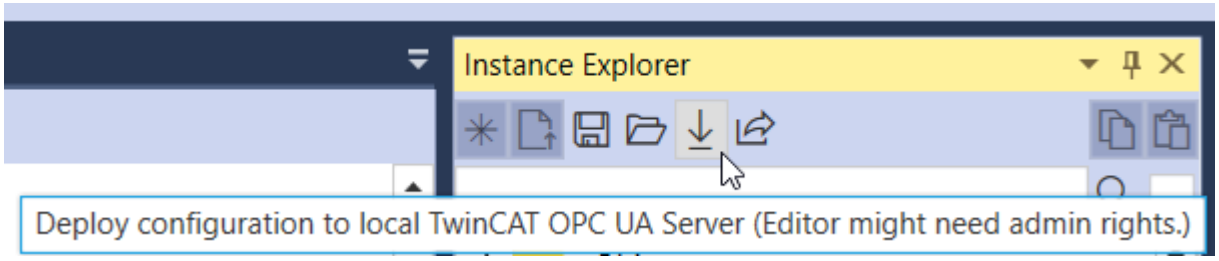
The software then starts.

4.9 Exporting a project

The user interface offers two options for exporting and using the current project with a TwinCAT OPC UA Server. The first option for the **local server** is primarily designed for the “same system” scenario described in the Use cases [▶ 61]. The second option for **non-local servers** is primarily intended for the "Different systems" scenarios.

Deployment of the project on the local TwinCAT OPC UA server

To deploy the project to the local TwinCAT OPC UA Server, click the **Deploy configuration to local TwinCAT OPC UA Server** button in the Instance Explorer toolbar.



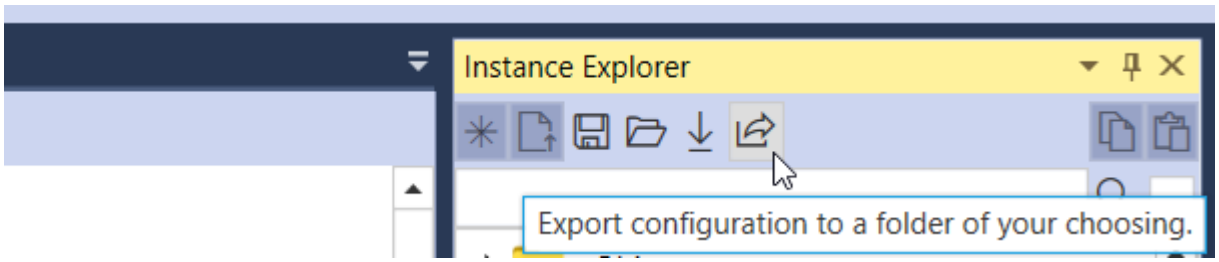
This exports the project nodeset, including the necessary dependencies, to the locally installed TwinCAT OPC UA Server and also creates a corresponding configuration file for the server. The directory from the options or the project file is used as the target (see Application and project settings [▶ 43]).

Restart the TwinCAT OPC UA Server to load the configuration

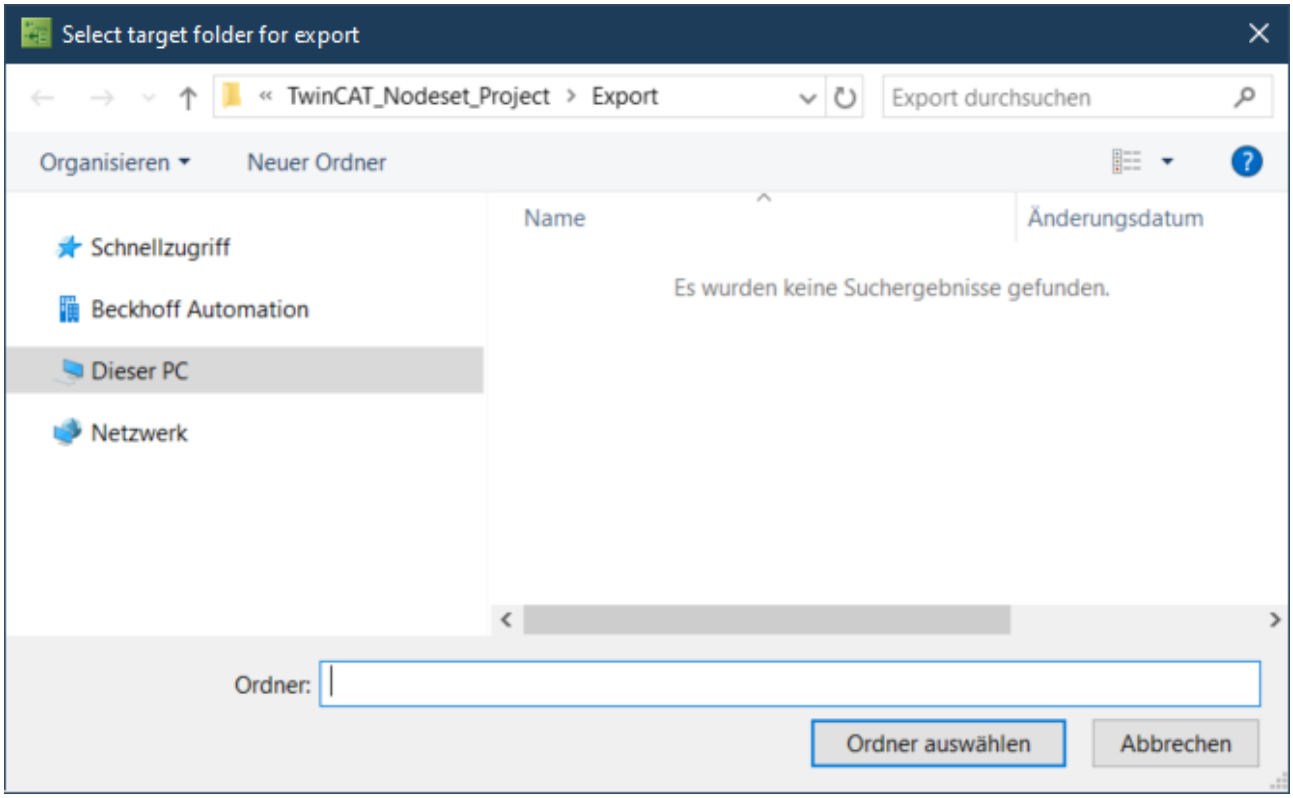
The TwinCAT OPC UA Server must be restarted after the configuration has been adjusted in order to apply it. For this reason, the server should be restarted after deployment via the Nodeset Editor in order to load the new TcUaNodesetConfig and the nodesets configured in it. The address space of the server then contains all the nodes contained in the nodesets.

Deployment of the project on a non-local TwinCAT OPC UA Server

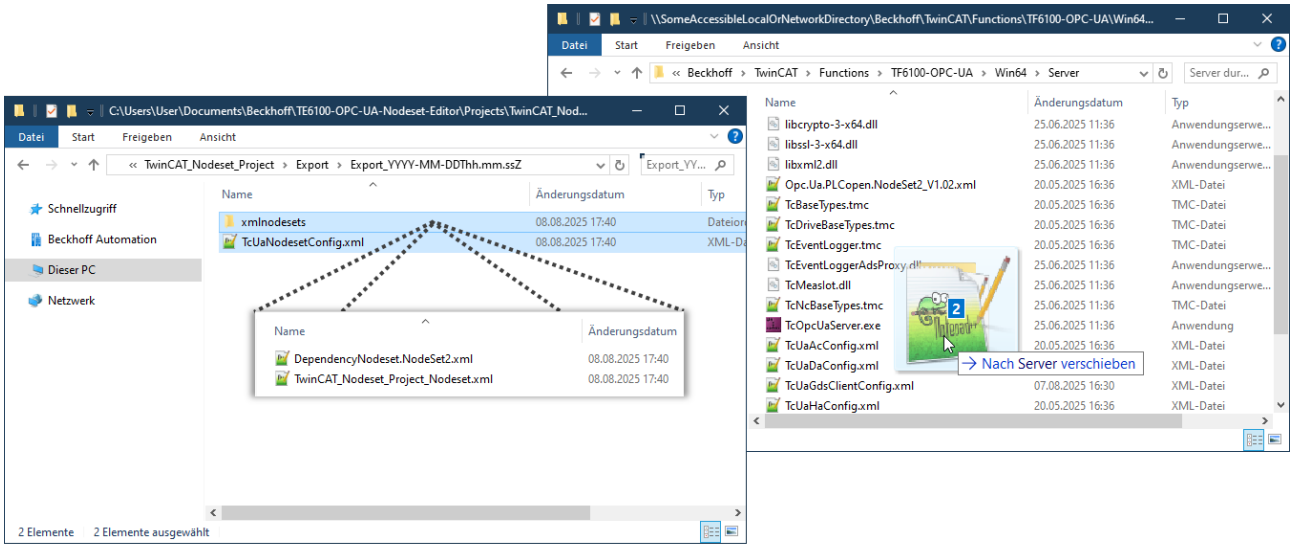
To deploy the project to a TwinCAT OPC UA Server that is not installed locally, click on the **Export configuration to a folder of your choosing** button in the Instance Explorer toolbar.



A selection dialog opens, which opens in the export directory (see Application and project settings [▶ 43]) by default.



Once a folder has been selected, the project nodeset, including any necessary dependencies, is exported to this folder and a corresponding configuration file is created for the server. The entire contents of the folder can then be copied manually to a current TwinCAT 3 OPC UA Server (see Use cases [▶ 61]). If direct file access is possible, this can simply be done via Explorer, for example:



Restart the TwinCAT OPC UA Server to load the configuration

The TwinCAT OPC UA Server must be restarted after the configuration has been adjusted in order to apply it. For this reason, the server should be restarted after deployment via the Nodeset Editor in order to load the new TcUaNodesetConfig and the nodesets configured in it. The address space of the server then contains all the nodes contained in the nodesets.

If several TwinCAT OPC UA Servers are in use, the last step of the deployment can be repeated as often as required for non-local servers. Plans are in place for further deployment options in a future version of the editor that will simplify the transfer process to the TwinCAT OPC UA Server even more.

5 PLC API

5.1 Tc3_OpcUa

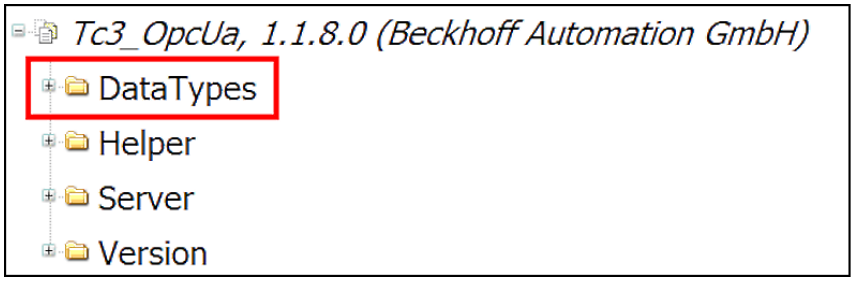
This PLC library contains data types, function blocks and more for use with TwinCAT OPC UA and consists of the following folders at the top level:

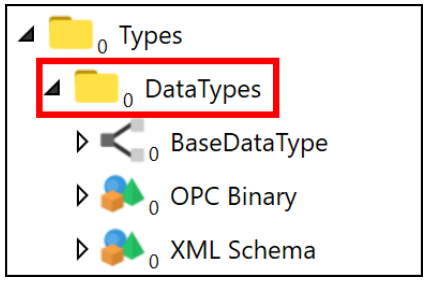
Table 1: Folder structure

Order name	Content description
DataTypes	Contains PLC types that map OPC UA data types (UA_DataType).
Helper	Contains PLC types that support the use of OPC UA data types.
Server	Contains PLC types (and more) for use with the TwinCAT OPC UA Server (TF6100).
Version	Contains version information from the PLC library.

5.1.1 Data types

This folder (left image) contains PLC types that map OPC UA data types (UA_DataType) and represents the node "DataTypes" [i=90] from the namespace http://opcfoundation.org/UA/ (right image).





5.1.1.1 BaseDataType

This folder represents the "BaseDataType" node [i=24] and contains PLC types that map its subtypes. BaseDataType itself does not have an equivalent PLC type.

5.1.1.1.1 UA_Boolean

This PLC data type maps the node "Boolean" [i=1]: reference.opcfoundation.org
This built-in data type defines a value that is either TRUE or FALSE.

Syntax

```
TYPE UA_Boolean : BOOL;
END_TYPE
```

5.1.1.1.2 UA_Byte

This PLC data type maps the node "Byte" [i=3]: reference.opcfoundation.org
This built-in data type defines a value that is either TRUE or FALSE.

Syntax

```
TYPE UA_Byte : USINT;
END_TYPE
```

5.1.1.1.3 UA_ByteString

This PLC data type maps the node "ByteString" [i=1]: reference.opcfoundation.org
This built-in data type defines a value that represents a sequence of byte values.

Syntax

```
TYPE UA_ByteString :
STRUCT
    nLength : UDINT;
    aData : ARRAY [0..255] OF BYTE;
END_STRUCT
END_TYPE
```

Parameter

Name	Type	Description
nLength	UDINT	Number of bytes used in aData
aData	ARRAY [0..255] OF BYTE	User data of the byte string

5.1.1.1.4 UA_DateTime

This PLC data type maps the node "DateTime" [i=13]: reference.opcfoundation.org
This built-in data type defines a date and time in the Gregorian calendar.

Syntax

```
TYPE UA_DateTime : DT;
END_TYPE
```

5.1.1.1.5 UA_Double

This PLC data type maps the node "Double" [i=11]: reference.opcfoundation.org
This built-in data type defines a floating point value according to the ISO/IEC/IEEE 60559:2020 definition with double accuracy (64 bits).

Syntax

```
TYPE UA_Double : LREAL;
END_TYPE
```

5.1.1.1.6 UA_Float

This PLC data type maps the node "Float" [i=10]: reference.opcfoundation.org
This built-in data type defines a floating point value according to the ISO/IEC/IEEE 60559:2020 definition with single accuracy (32 bits).

Syntax

```
TYPE UA_Float : REAL;
END_TYPE
```

5.1.1.1.7 UA_Int16

This PLC data type maps the node "Int16" [i=4]: reference.opcfoundation.org
This built-in data type defines a signed integer between -32,768 and 32,767.

Syntax

```
TYPE UA_Int16 : INT;
END_TYPE
```

5.1.1.1.8 UA_Int32

This PLC data type maps the node "Int32" [i=6]: reference.opcfoundation.org
This built-in data type defines a signed integer between -2,147,483,648 and 2,147,483,647.

Syntax

```
TYPE UA_Int32 : DINT;
END_TYPE
```

5.1.1.1.9 UA_Int64

This PLC data type maps the node "Int64" [i=8]: reference.opcfoundation.org
This built-in data type defines a signed integer between -9,223,372,036,854,775,808 and 9,223,372,036,854,775,807.

Syntax

```
TYPE UA_Int64 : LINT;
END_TYPE
```

5.1.1.1.10 UA_LocaleId

This PLC data type maps the node "LocaleId" [i=295]: reference.opcfoundation.org
This simple data type is a character string consisting of a language code and an optional country code in accordance with RFC 5646 (e.g. "de-DE").

Syntax

```
TYPE UA_LocaleId : STRING(6);
END_TYPE
```

5.1.1.1.11 UA_LocalizedText

This PLC data type maps the node "LocalizedText" [i=21]: reference.opcfoundation.org
This built-in data type defines a structure consisting of a string/text and a locale identifier for localized translations.

Syntax

```
{attribute 'pack_mode' := '1'}
TYPE UA_LocalizedText :
STRUCT
    sLocaleId : UA_LocaleId;
    {attribute 'TcEncoding' := 'UTF-8'}
    sText : STRING(256);
END_STRUCT
END_TYPE
```

Parameters

Name	Type	Description
sLocaleId	Tc3_OpcUa.UA_LocaleId	Locale identifier
sText	STRING(256)	Localized string/text

5.1.1.1.12 UA_NodeId

This PLC data type maps the node "NodeId" [i=17]: reference.opcfoundation.org
This built-in data type consists of three elements (namespace index, ID type, identifier) that uniquely identify a node within a server.

Syntax

```
{attribute 'pack_mode' := '1'}
TYPE UA_NodeId :
STRUCT
    xml : UA_String;
END_STRUCT
END_TYPE
```

Parameter

Name	Type	Description
xml	Tc3_OpcUa.UA_String	Complete NodeId encoded as a string e.g. ns=5;i=1234

5.1.1.1.13 UA_SByte

This PLC data type maps the node "SByte" [i=2]: reference.opcfoundation.org
This built-in data type defines a signed integer between -128 and 127.

Syntax

```
TYPE UA_SByte : SINT;
END_TYPE
```

5.1.1.1.14 UA_String

This PLC data type maps the node "String" [i=12]: reference.opcfoundation.org
This built-in data type defines a Unicode string that should not contain any control characters other than spaces.

Syntax

```
TYPE UA_String : STRING;
END_TYPE
```

5.1.1.1.15 UA_UInt16

This PLC data type maps the node "UInt16" [i=5]: reference.opcfoundation.org
This built-in data type defines an unsigned integer between 0 and 65,535.

Syntax

```
TYPE UA_UInt16: UINT;
END_TYPE
```

5.1.1.1.16 UA_UInt32

This PLC data type maps the node "UInt32" [i=7]: reference.opcfoundation.org
This built-in data type defines an unsigned integer between 0 and 4,294,967,295.

Syntax

```
TYPE UA_UInt32 : UDINT
END_TYPE
```

5.1.1.1.17 UA_UInt64

This PLC data type maps the node "UInt64" [i=9]: reference.opcfoundation.org
This built-in data type defines an unsigned integer between 0 and 18,446,744,073,709,551,615.

Syntax

```
TYPE UA_UInt64 : ULINT
END_TYPE
```

5.1.1.1.18 UA_UtcTime

This PLC data type maps the node "UtcTime" [i=294]: reference.opcfoundation.org
This simple data type is a DateTime value for representing coordinated universal time (UTC), which is used exclusively in OPC UA communication.

Syntax

```
TYPE UA_UtcTime : UA_DateTime
END_TYPE
```

5.1.1.1.19 Structure

This folder represents the "Structure" node [i=22] and contains PLC types that map its subtypes. "Structure" itself does not have an equivalent PLC type. Concrete OPC UA structure types derived from "Structure" can be mapped as PLC data types:

Mapping of an OPC UA structure type as a PLC data type:

Requirements

1. The data structure in the PLC matches the OPC UA data structure. In particular:
   - Sequence of fields
   - Data types
2. The PLC type definition contains the OPC UA type information as attributes.
   The TwinCAT OPC UA Server requires these attributes in order to assign the PLC data type to the correct OPC UA data type. Simple PLC data types do not require these attributes.

The required attributes can be written manually above the instance or type declaration or retrieved for copying via the TwinCAT OPC UA Nodeset Editor.

Step-by-step with the Nodeset Editor:

1 - Finding and opening the structure type.

5 - Creating the DUT in TwinCAT.

2 – Selection and copying of all names and DataType BrowseNames of the fields

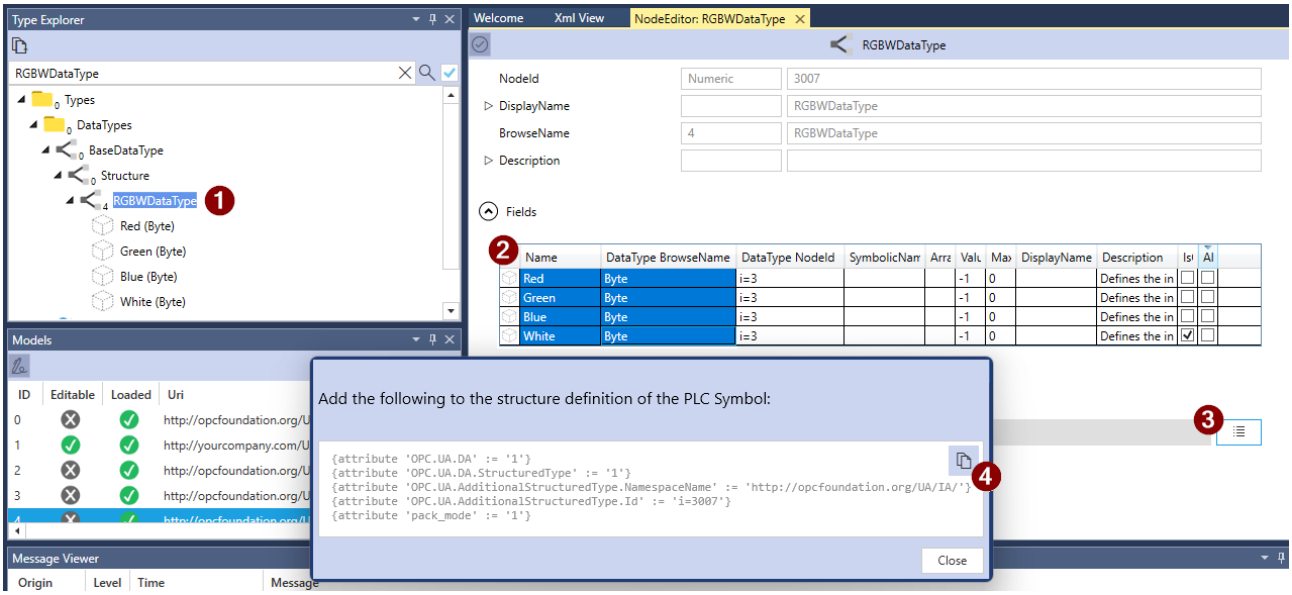
5 - Insert in TwinCAT between STRUCT and END_STRUCT

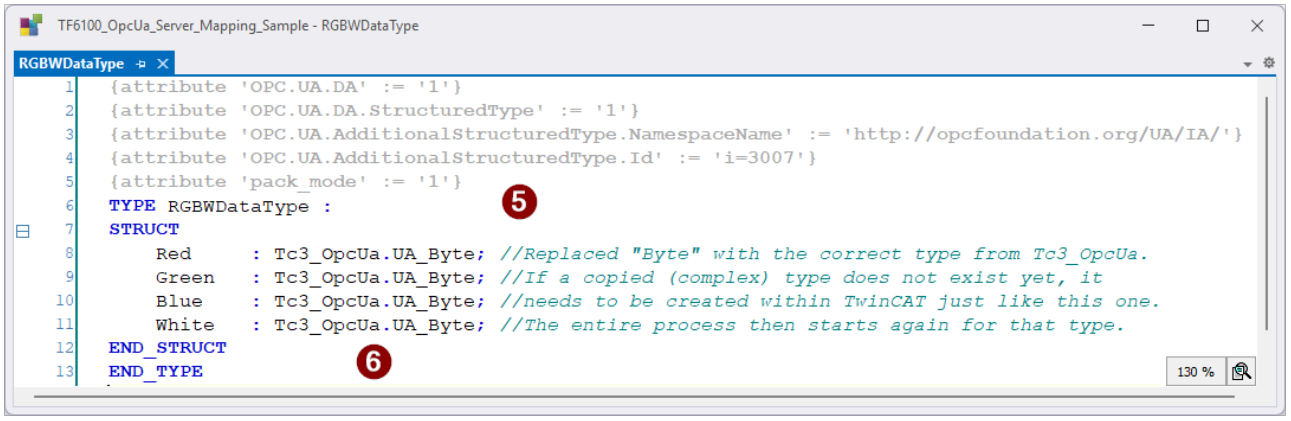
3 - Opening of the pragmas for the type

4 - Copying the pragmas

5 - Inserting in TwinCAT above TYPE

6 - Replacing the DataType BrowseNames with the appropriate PLC types
(e.g. Boolean => "Tc3_OpcUa.UA_Boolean" or "BOOL").
Missing (structure) types may also need to be created.





Meaning of the attributes:

- **OPC.UA.DA**
  Activates the provision of the data type and any sub-elements via OPC UA.
- **OPC.UA.DA.StructuredType**
  Identifies the type as a structured OPC UA data type.
- **OPC.UA.AdditionalStructuredType.NamespaceName**
  Specifies the NamespaceUri of the data type from the nodeset.
- **OPC.UA.AdditionalStructuredType.Id**
  Contains the NodeId of the OPC UA data type, in this case i=3007.
- **pack_mode := '1'**
  Ensures that the memory layout of the structure is defined and compatible.

5.1.1.1.19.1 UA_EUInformation

This PLC data type maps the node "EUInformation" [i=887]: reference.opcfoundation.org
This structured data type contains information about units of measurement (e.g. namespace URI, unit ID, display name and description).

Syntax

```
{attribute 'pack_mode' := '1'}
TYPE UA_EUInformation :
STRUCT
    sNamespaceURI:  UA_String;
    nUnitId:        UA_Int32;
    sDisplayName:   UA_LocalizedText;
    sDescription:   UA_LocalizedText;
END_STRUCT
END_TYPE
```

Parameter

Name	Type	Description
sNamespaceURI	Tc3_OpcUa.UA_String	URI of the organization that defines this unit of measurement
nUnitId	Tc3_OpcUa.UA_Int32	Numerical identifier for programmatic evaluation
sDisplayName	Tc3_OpcUa.UA_LocalizedTExt	Usually abbreviation of the unit of measurement
sDescription	Tc3_OpcUa.UA_LocalizedTExt	Full (written in full) name of the unit of measurement

5.1.1.1.19.2 UA_OptionSet

This PLC data type maps the "OptionSet" node [i=12755]: reference.opcfoundation.org
This abstract data type is the base type for all data types that represent a bit mask with more than 64 bits or where the validity of individual bits must be marked.

Syntax

```
TYPE UA_OptionSet :
STRUCT
END_STRUCT
END_TYPE
```

Using UA_OptionSet

To define a specific OptionSet and use it with TF6100 (Server) and TE6100 (Nodeset Editor), it must be derived from Tc3_OpcUa.UA_OptionSet.

Sample implementation of a specific option set:

```
TYPE UA_OptionSet_Sample EXTENDS UA_OptionSet :
STRUCT
    BitOne : UA_OptionSet_Option;
    BitTwo : UA_OptionSet_Option;
END_STRUCT
END_TYPE
```

5.1.1.1.19.2.1 UA_OptionSet_Option

This PLC data type is an auxiliary data type for defining your own OptionSet data types.
This data type maps an option (a bit) in an OptionSet (a bit mask) including a validity specification.

Syntax

```
TYPE UA_OptionSet_Option :
STRUCT
    value : BOOL;
    valid : BOOL;
END_STRUCT
END_TYPE
```

Parameter

Name	Type	Description
value	BOOL	Value of this option
Valid	BOOL	Validity specification of this option

5.1.1.1.19.3 UA_Range

This PLC data type maps the "Range" node [i=884]: reference.opcfoundation.org
This structured data type defines a value range with a lower (low) and an upper (high) double value.

Syntax

```
{attribute 'pack_mode' := '1'}
TYPE UA_Range :
STRUCT
    fLow : UA_Double;
    fHigh: UA_Double;
END_STRUCT
END_TYPE
```

Parameter

Name	Type	Description
fLow	Tc3_OpcUa.UA_Double	Lower limit of the value range
fHigh	Tc3_OpcUa.UA_Double	Upper limit of the value range

5.1.1.1.19.4 UA_TimeZoneDataType

This PLC data type maps the node "TimeZoneDataType" [i=8912]: reference.opcfoundation.org
This structured data type defines the local time, which may or may not take daylight saving time into account.

Syntax

```
{attribute 'pack_mode' := '1'}
TYPE UA_TimeZoneDataType :
STRUCT
    nOffset                : UA_Int16;
    bDaylightSavingInOffset : UA_Boolean;
END_STRUCT
END_TYPE
```

Parameter

Name	Type	Description
nOffset	Tc3_OpcUa.UA_Int16	Time offset in minutes relative to UTC time
nDaylightSavingInOffset	Tc3_OpcUa.UA_Boolean	Indication of whether the offset includes the summer time correction

5.1.2 Function blocks

This folder contains PLC function blocks for use with TwinCAT OPC UA.

5.1.2.1 Server

This folder contains PLC function blocks for use with the TwinCAT OPC UA Server (TF6100).

5.1.2.1.1 FB_OpcUaServer_SetServiceLevel



The TwinCAT OPC UA Server currently sets the ServiceLevel statically to the value 255. This function block enables the ServiceLevel to be set via an ADS interface in the server. A distinction is made between use with and without a watchdog.

Further information on the service level can be found in the ServiceLevel Documentation of the TwinCAT OPC UA Server.

Syntax

```
FUNCTION BLOCK FB_OpcUaServer_SetServiceLevel
VAR_INPUT
    tAmsNetId              : T_AmsNetId := '127.0.0.1.1.1';
    tTimeout               : TIME := T#5s;;
    bExecute               : BOOL;
    bWatchdog              : BOOL;
    nServiceLevel          : BYTE;
END_VAR
VAR_OUTPUT
    bBusy                  : BOOL;
    bError                 : BOOL;
    nErrorId               : UDINT;
END_VAR
```

Inputs

Name	Type	Description
tAmsNetId	T_AmsNetId	AmsNetId of the system on which the TwinCAT OPC UA Server runs. If no information is provided, the local host is used.
tTimeout	TIME	Maximum time for executing the ADS command. The setting is 5 seconds by default.
bExecute	BOOL	Optional parameter to specify a URI for the WebSocket opening handshake.
bWatchdog	BOOL	Determines whether the ServiceLevel is set with or without a watchdog.
nServiceLevel	BYTE	ServiceLevel that is set in the TwinCAT OPC UA Server.

Outputs

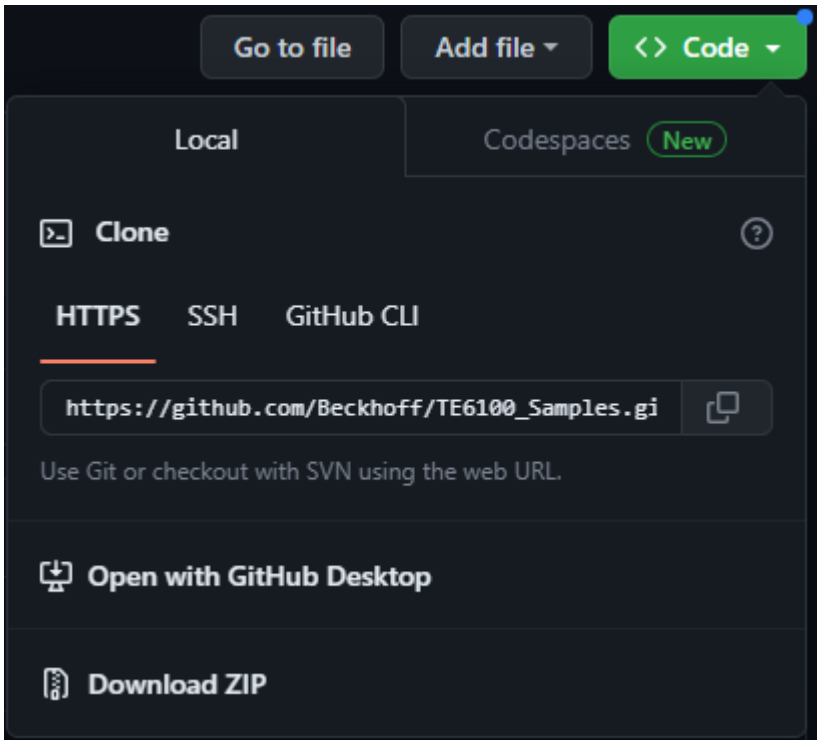
Name	Type	Description
bBusy	BOOL	Is TRUE as long as the function block is busy with processing.
bError	BOOL	Becomes TRUE when an error situation occurs.
nErrorId	UDINT	ADS error code.

Requirements

Development environment	Target platform	PLC libraries to include
TwinCAT v3.1.4026.x	IPC or CX (x86, x64, Arm®)	Tc3_OpcUa (1.1.8 or higher)

6 Samples

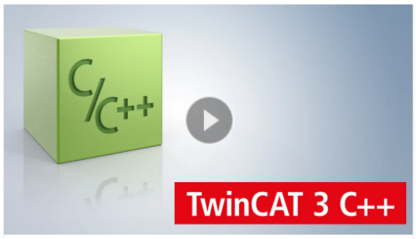
Sample code and configurations for this product can be obtained from the corresponding repository on GitHub: https://github.com/Beckhoff/TE6100_Samples. There you have the possibility to clone the repository or download a ZIP file with the sample.



We also recommend to check out our quick start tutorials [▶ 14], which provide a guided walkthrough of various configuration use cases.

7 Tutorials

You can find video tutorials for this product on our website at https://www.beckhoff.com/tutorials. The video tutorials provide a quick introduction to the product and the individual product facets.



14.03.2024 | Multimedia
Tutorial: TwinCAT 3 C++ overview
Get an overview of the C++ integration into TwinCAT.
Learn more →



12.02.2024 | Multimedia
Tutorial: Chart synchronization in TwinCAT Scope
Learn how to dock and synchronize charts in TwinCAT Scope.
Learn more →



12.02.2024 | Multimedia
Tutorial: Getting started with TF6100 TwinCAT 3 OPC UA Server
Learn how to configure the TwinCAT OPC UA Server.
Learn more →

8 Appendix

8.1 Terminology

Nodeset

A nodeset is a file that formally describes an OPC UA information model. Nodeset files are usually XML-based and must be structured according to an XML schema definition. Nodesets are used for processing information models with the computer. A nodeset defines nodes, their attributes, and references. A reference can point to nodes in the same document as well as to nodes outside the file. OPC UA itself defines some central information models with corresponding nodesets, for example:

Name	Namespace
Core („Base“)	http://opcfoundation.org/UA
DI	http://opcfoundation.org/UA/DI
GDS	http://opcfoundation.org/UA/GDS

Particularly noteworthy is the "Core" model, whose nodeset is also colloquially referred to as the "Base" nodeset. The core model usually serves as a basis for other information models and describes the basic data types that are available in OPC UA.

Companion specifications

Although the existing OPC UA information models are already applicable in many industries, it is possible to create own information models to meet specific requirements. For this purpose, companies and organizations join forces to define industry-specific models, so-called "companion specifications". Such companion specifications are available, for example, for the packaging industry (PackML) or for plastics processing (EUROMAP).

Further information on this topic can be found on the OPC Foundation website.

Node

A node is a fundamental part of the address space of a server and represents an object, variable or method. Even data and object types are represented on the server’s address space as a node.

8.2 Troubleshooting

Behavior	Description
The configuration cannot be exported to the TwinCAT OPC UA Server. The log shows an error “Export failed. Reason: ...".	• Please make sure that you have write access to the TwinCAT OPC UA Server installation directory. If the TwinCAT OPC UA Server has been installed to the %ProgramFiles% directory, please make sure to start the TwinCAT OPC UA Nodeset Editor as Administrator using the “Run As Administrator” option.
I have exported my configuration to the TwinCAT OPC UA Server, but my nodes are not visible in the server address space.	• Please make sure that you have restarted the server after the export. This is the only way for the server to process the new configuration. • If the behavior persists, please make sure that you have configured the correct installation path for the TwinCAT OPC UA Server in the Application settings [▶ 43]. On a 32 bit system, the Win32 installation directory is used, while on a 64 bit system, the Win64 directory is used to run the TcOpcUaServer.exe file. You can also double-check which directory used via the Windows Task Manager by navigating to the TcOpcUaServer process and selecting Open file location from the context menu.

8.3 Usage scenarios and version compatibility

Table 2: Usage scenarios

Scenario	Modeling system TwinCAT version	Target system TwinCAT version	Installation Editor (TE6100) [▶ 10]	Installation Server (TF6100)	Purpose
Same system	4026 XAE and XAR		Local with Package Manager		Development, productive use
Different systems	4026 XAE	4026 XAR	On XAE system with Package Manager	On XAR system with Package Manager	Development, productive use
Different systems legacy	4026 XAE and 4024 Remote Manager	4024 XAR	On XAE system with Package Manager	On XAR system with TwinCAT 3 setup	Maintenance of old systems

As long as the Nodeset Editor is used on a system with the TwinCAT Package Manager and thus TwinCAT Build 4026, the TwinCAT OPC UA Server can also be installed on another system that meets its system requirements.

If the server is only available in an old version that cannot be changed, the following compatibility matrix applies:

Table 3: Version compatibility

Server version	Editor version	Functionality
>= 5.2	>= 2	✓ Full
5.1	1	⚠ Restricted, not recommended
< 5.1	-	✗ None
Further combinations		✗ None

The rest of the documentation assumes that the current Nodeset Editor and the current server are installed as in the "Same system" scenario.

9 Support and Service

Beckhoff and their partners around the world offer comprehensive support and service, making available fast and competent assistance with all questions related to Beckhoff products and system solutions.

Download finder

Our download finder contains all the files that we offer you for downloading. You will find application reports, technical documentation, technical drawings, configuration files and much more.

The downloads are available in various formats.

Beckhoff's branch offices and representatives

Please contact your Beckhoff branch office or representative for local support and service on Beckhoff products!

The addresses of Beckhoff's branch offices and representatives round the world can be found on our internet page: www.beckhoff.com

You will also find further documentation for Beckhoff components there.

Beckhoff Support

Support offers you comprehensive technical assistance, helping you not only with the application of individual Beckhoff products, but also with other, wide-ranging services:

- support
- design, programming and commissioning of complex automation systems
- and extensive training program for Beckhoff system components

Hotline:	+49 5246 963-157
e-mail:	support@beckhoff.com

Beckhoff Service

The Beckhoff Service Center supports you in all matters of after-sales service:

- on-site service
- repair service
- spare parts service
- hotline service

Hotline:	+49 5246 963-460
e-mail:	service@beckhoff.com

Beckhoff Headquarters

Beckhoff Automation GmbH & Co. KG

Huelshorstweg 20
33415 Verl
Germany

Phone:	+49 5246 963-0
e-mail:	info@beckhoff.com
web:	www.beckhoff.com

Trademark statements

Beckhoff®, ATRO®, EtherCAT®, EtherCAT G®, EtherCAT G10®, EtherCAT P®, MX-System®, Safety over EtherCAT®, TC/BSD®, TwinCAT®, TwinCAT/BSD®, TwinSAFE®, XFC®, XPlanar® and XTS® are registered and licensed trademarks of Beckhoff Automation GmbH.

Third-party trademark statements

Arm, Arm9 and Cortex are trademarks or registered trademarks of Arm Limited (or its subsidiaries or affiliates) in the US and/or elsewhere.

Excel, IntelliSense, Microsoft, Microsoft Azure, Microsoft Edge, PowerShell, Visual Studio, Windows and Xbox are trademarks of the Microsoft group of companies.

OPC UA is a registered trademark of the OPC Foundation.

More Information:
www.beckhoff.com/te6100

Beckhoff Automation GmbH & Co. KG
Hülshorstweg 20
33415 Verl
Germany
Phone: +49 5246 9630
info@beckhoff.com
www.beckhoff.com